

RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du

Diplôme National en Sciences Informatiques et Web

Spécialité : Génie Logiciel et Systèmes d'Information

Par

Cherif Omar et **Saadaoui Safa**

Développement d'une application web pour l'évaluation des risques d'incendies forestiers par data science spatiale dans les régions de Kroumirie

Encadrant professionnel : **Monsieur Benzarti Hassen**

Lead développeur

Encadrant académique : **Monsieur Toujani Ahmed**

Maître Assistant(e)

Réalisé au sein de Medianet



Année universitaire : 2024 - 2025

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Monsieur Benzarti Hassen**

Signature et cachet

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Monsieur Toujani Ahmed**

Signature

Dédicaces

Ce mémoire est bien plus qu'un simple aboutissement académique ; il est l'expression d'un parcours humain riche de rencontres, de défis et de partages.

Je dédie ce travail à ceux qui ont jalonné ce chemin avec bienveillance, exigence et inspiration.

À **Monsieur Toujani Ahmed** et **Monsieur Benzarti Hassen** , pour la confiance qu'ils nous ont témoignée, leur accompagnement attentif, et leur volonté constante de nous faire progresser. Leur rigueur et leur passion ont été de véritables moteurs dans cette aventure.

À **Madame Nadia Esseghir**, pour son soutien moral précieux et ses encouragements constants tout au long de ce projet. Sa générosité d'esprit, sa bienveillance et sa présence réconfortante ont été une source de motivation inestimable dans les moments les plus exigeants.

À nos familles, véritables piliers de notre vie, pour leur amour inconditionnel, leur patience et leurs encouragements silencieux mais puissants. Chaque pas franchi dans ce projet leur doit beaucoup.

Aux amis fidèles, aux épaules disponibles, aux sourires rassurants : merci d'avoir été là, dans les moments de doute comme dans les instants de joie.

Enfin, à toutes celles et ceux qui, par un mot, un geste, une présence, ont contribué à rendre ce parcours plus lumineux : cette réussite est aussi la vôtre.

Cherif Omar et Saadaoui Safa

Remerciements

« La gratitude est la mémoire du cœur. » – Jean-Baptiste Massieu

Au terme de ce projet de fin d'études, nous souhaitons exprimer notre profonde reconnaissance à toutes les personnes qui ont, de près ou de loin, contribué à la concrétisation de ce travail, lequel marque l'aboutissement d'un parcours académique et humain enrichissant. Nous exprimons notre gratitude la plus sincère à Monsieur **Hassen Benzarti**, notre encadrant professionnel, pour son accompagnement rigoureux, sa vision pragmatique et ses conseils éclairés qui ont été essentiels à la réalisation de ce projet. Son exigence professionnelle et son sens de la transmission ont été pour nous une source d'inspiration. Nos remerciements s'adressent également à Monsieur **Ahmed Toujani**, notre encadrant académique, pour sa bienveillance, sa rigueur scientifique et la qualité de son suivi tout au long de cette expérience. Son regard critique et constructif a enrichi notre réflexion et renforcé la valeur de notre travail.

« Se réunir est un début, rester ensemble est un progrès, travailler ensemble est la réussite. » – Henry Ford

Ce projet est le fruit d'un travail de collaboration harmonieuse entre nous, **Chérif Omar** et **Saâdaoui Safa**. À travers nos différences et nos complémentarités, nous avons su conjuguer nos efforts, nos compétences et notre détermination dans une dynamique commune, portée par le respect, l'écoute et l'engagement. Nous ne saurions conclure sans remercier chaleureusement nos familles, qui ont toujours cru en nous et nous ont soutenus avec patience, amour et encouragement. Leur présence, silencieuse mais constante, a été un véritable pilier tout au long de ce parcours.

Enfin, nous adressons nos pensées les plus sincères à nos amis et proches, pour leur soutien moral, leurs encouragements et leur confiance, qui ont largement contribué à maintenir notre motivation et notre persévérance.

« La reconnaissance est un devoir, et celui qui oublie ses bienfaiteurs trahit une part de lui-même. » – Sénèque

Table des matières

Liste des abréviations	x
Introduction générale	1
1 Cadre théorique et technique	2
1.1 Incendies de forêt : contexte et enjeux	3
1.1.1 Définition des incendies de forêt	3
1.1.2 Enjeux écologiques, économiques et sociaux	3
1.1.3 Exemples récents	4
1.2 Systèmes d'Information Géographique (SIG)	4
1.2.1 Types de données	4
1.2.2 Applications des SIG dans les incendies	4
1.3 Web mapping et TerriaJS	4
1.4 Data science appliquée aux données spatiales	5
1.4.1 Notions de base	5
1.4.2 Exemples d'analyse	5
1.5 Outils et technologies utilisés	5
1.5.1 Les Environnements Matériels	5
1.5.2 Outils et technologies utilisés	5
2 Zone d'étude et données	8
2.1 Présentation de la Kroumirie	9
2.1.1 Caractéristiques physiques et écologiques	9
2.1.2 Contexte socio-économique	9
2.1.3 Vulnérabilité aux incendies	10
2.2 Données spatiales collectées	10
2.2.1 Cartes vecteurs initiales	10
2.2.2 Cartes raster existantes	14
2.3 Conversion des vecteurs en rasters	20
2.3.1 Intérêt de la conversion	20
2.3.2 Méthodologie	20
2.4 Données d'incendies	21

2.4.1	Sources des données d'incendies	21
2.4.2	Données météorologiques associées	22
2.5	Pré-traitement des données raster	23
3	Modélisation et développement du système	24
3.1	Justification du modèle en cascade	25
3.2	Diagrammes de modélisation	25
3.2.1	Diagramme des phases (cascade)	25
3.2.2	Diagrammes UML	25
3.2.3	Architecture générale du système	35
3.3	Développement backend (Laravel)	36
3.3.1	Structure du code	37
3.3.2	API RESTful	37
3.3.3	Sécurité et authentification	37
3.3.4	Interactions avec la base de données spatiale	38
3.3.5	Fonctionnalités serveur implémentées	38
3.4	Développement frontend (React & TerriaJS)	39
3.4.1	Architecture de l'interface utilisateur	39
3.4.2	Intégration de TerriaJS pour la visualisation cartographique	39
3.4.3	Justification du choix de TerriaJS	40
3.4.4	Communication avec le backend	40
3.4.5	Fonctionnalités principales développées côté interface	41
3.5	Extraction des valeurs raster pour la data science	41
3.5.1	Objectif de l'extraction raster	41
3.5.2	Méthodologie générale	42
3.5.3	Couches raster utilisées	42
3.5.4	Exemple de fonction d'extraction	42
3.5.5	Avantages de l'approche	44
3.5.6	Résumé des scripts	44
3.6	Modélisation Data Science	45
3.6.1	Construction de la base d'apprentissage	45
3.6.2	Méthodes utilisées	46
3.6.3	Évaluation des modèles	47
3.7	Résultats de la modélisation	47

3.7.1	Cartes des zones à risque (Classification par cluster)	47
3.7.2	Cartes de prédiction de surface brûlée (Régression par cluster)	50
3.8	Intégration des résultats dans le web mapping	52
3.8.1	3.8 Intégration des résultats dans le web mapping	52
3.9	Visualisation des interfaces utilisateur	54
4	Discussion	60
4.1	Analyse des résultats	61
4.2	Limites de l'approche	61
4.3	Pistes d'amélioration	62
	Conclusion générale	63
	Annexes	66
A	ANNEXE A : PostGIS et gestion des couches spatiales	67
A.1	PostGIS et gestion des couches spatiales	68
B	ANNEXE B :Requêtes spatiales SQL utilisées dans le projet	72
B.1	Requêtes spatiales SQL utilisées dans le projet	73
C	ANNEXE C :Intégration du modèle de prédiction via API Flask	76
C.1	Intégration du modèle de prédiction via API Flask	77

Table des figures

1	Carte des séries forestières de la Kroumirie (gouvernorat de Jendouba, polygones) . . .	11
2	Couche vecteur : Séries forestières de Jendouba(polygones)	12
3	Couche vecteur : Parcelles forestières de Jendouba(polygones)	12
4	Couche vecteur : Réseau routier(lignes)	13
5	Couche vecteur : Barrages collinaires (points)	13
6	Couche vecteur : Villages (points)	14
7	Couche raster : Barrages collinaires (ponctuels)	15
8	Couche raster : Hydrographie linéaire	16
9	Couche raster : Infrastructure hydraulique ponctuelle – lacs collinaires	16
10	Couche raster : Infrastructures Hydrauliques Surfaciues – Grands barrages	17
11	Couche raster : Infrastructures Hydrauliques Surfaciues	17
12	Couche raster : Lacs collinaires	18
13	Couche raster : Périmètres irrigués	18
14	Couche raster : Réseau routier	19
15	Couche raster : Unités de transformation agricole	19
16	Couche raster : Zones urbaines (terrains construits)	20
17	Répartition géographique des incendies dans la région de Jendouba (données 2000–2024)	22
18	Diagramme de cas d'utilisation	26
19	Diagramme de classes du système	31
20	Diagramme de séquence — Authentification	32
21	Diagramme de séquence — Création d'un utilisateur	33
22	Diagramme de séquence — Distribution d'une série	34
23	Diagramme de séquence — Déclaration d'un incendie	35
24	Architecture horizontale du système basé sur le modèle MVC	36
25	Extrait des routes API définies dans <code>routes/api.php</code>	37
26	Extrait des routes API sécurisées définies dans <code>routes/api.php</code>	38
27	Exemple de requête spatiale avec PostGIS dans l'IDE	38
28	Fonction Python <code>get_raster_value</code> pour extraire une valeur raster à partir des coordonnées.	43
29	Fonction <code>update_incendie</code> pour mettre à jour dynamiquement les attributs raster dans la base de données.	43

30	Fonction <code>enrich_dgf.py</code> pour enrichir tous les incendies de la table <code>dgf</code>	44
31	Carte des zones à risque d'incendie – Cluster 0 (Jendouba)	48
32	Carte des zones à risque d'incendie – Cluster 1 (Jendouba)	48
33	Carte des zones à risque d'incendie – Cluster 2 (Jendouba)	49
34	Carte estimée de la surface brûlée (ha) – Cluster 0 (Jendouba)	50
35	Carte estimée de la surface brûlée (ha) – Cluster 1 (Jendouba)	51
36	Carte estimée de la surface brûlée (ha) – Cluster 2 (Jendouba)	51
37	Architecture technique corrigée entre React, Laravel et Flask	53
38	Interface de connexion de l'utilisateur	55
39	Tableau de bord du visiteur	55
40	Interface forestier avec déclaration d'incendie et carte des séries	56
41	Tableau de bord de l'ingénieur avec prédiction et visualisation raster	56
42	Interface de gestion des utilisateurs par l'administrateur	57
43	Carte interactive affichant un incendie et une couche raster	57
44	Formulaire de prédiction météo et affichage du résultat sur la carte	58
45	Popup d'incendie avec valeur prédite et légende associée	59
46	Script Bash automatisant l'importation des couches raster dans PostGIS	69
47	Script Bash pour l'importation automatique de couches vecteur (GeoJSON) dans PostGIS	70
48	Serveur Flask – prédiction de cluster	78
49	Requête vers Flask et récupération du cluster	78

Liste des tableaux

1	Environnement Matériel	6
2	Comparaison des données d’incendies pour la région de Jendouba selon trois sources .	21
3	Description textuelle du cas d’utilisation « Authentification »	27
5	Description textuelle du cas d’utilisation « Déclarer un incendie »	28
7	Description textuelle du cas d’utilisation « Attribuer les séries forestières »	29
9	Liste des scripts d’extraction raster développés	44
10	Résumé des figures associées à l’intégration web mapping	54

Liste des abréviations

- **API** = **A**pplication **P**rogramming **I**nterface – Interface de programmation applicative
- **DGF** = **D**irection **G**énérale des **F**orêts – Source nationale de données sur les incendies
- **EPSG** = **E**uropean **P**etroleum **S**urvey **G**roup – Code de référence pour les systèmes de coordonnées
- **GIS** = **G**eographic **I**nformation **S**ystem – Système d’information géographique (SIG)
- **GeoJSON** = **G**eographical **J**SON – Format d’échange de données spatiales au format JSON
- **GeoTIFF** = **G**eographical **T**agged **I**mage **F**ile **F**ormat – Format raster géoréférencé
- **Leaflet** = Bibliothèque JavaScript pour la cartographie web interactive
- **NASA** = **N**ational **A**eronautics and **S**pace **A**dministration – Agence spatiale américaine
- **PostGIS** = Extension spatiale de PostgreSQL permettant la gestion de données géographiques
- **QGIS** = **Q**uantum **G**IS – Logiciel libre de SIG bureautique
- **REST** = **R**epresentational **S**tate **T**ransfer – Architecture d’API web
- **SQL** = **S**tructured **Q**uery **L**anguage – Langage de requête relationnel
- **UTM** = **U**niversal **T**ransverse **M**ercator – Système de projection cartographique utilisé

Introduction générale

Dans un contexte de changement climatique et d'intensification des phénomènes extrêmes, les incendies de forêts représentent une menace croissante pour les écosystèmes, les infrastructures et les populations. Face à ces enjeux, le recours à des outils numériques avancés devient indispensable pour renforcer les capacités de prévention, de surveillance et de gestion des risques. C'est dans ce cadre que s'inscrit le présent projet de fin d'études, réalisé au sein de l'entreprise **Medianet**, acteur reconnu dans le développement de solutions digitales innovantes.

L'objectif principal de ce projet est de concevoir et développer une application web interactive permettant d'évaluer les risques d'incendies forestiers à l'échelle régionale, en s'appuyant sur les données spatiales disponibles et les techniques de data science. La zone d'étude ciblée, la **Kroumirie**, située au nord-ouest de la Tunisie, est particulièrement vulnérable aux incendies en raison de sa richesse forestière, de son relief accidenté et de la pression anthropique qu'elle subit.

Pour répondre à cette problématique, l'application développée intègre plusieurs technologies modernes. Côté *backend*, le framework **Laravel** a été utilisé pour la gestion sécurisée des données et des utilisateurs. Côté *frontend*, l'interface a été conçue avec **React** et enrichie par **TerriaJS** pour la visualisation cartographique interactive. Les données géospatiales, traitées à l'aide de **PostGIS**, **Python** et **QGIS**, sont exploitées dans une approche analytique pour générer des modèles de **clustering** et de **régression** capables de prédire les zones à risque ou l'étendue des surfaces potentiellement brûlées.

La réalisation de ce projet a nécessité la mise en place d'une architecture modulaire, évolutive et performante, alliant rigueur scientifique et ergonomie d'usage. Elle démontre la pertinence de l'intégration des sciences des données dans le domaine de l'environnement et de la gestion des risques naturels.

Ce rapport est structuré comme suit :

- **Chapitre 1** : Présentation du cadre théorique et des outils technologiques mobilisés.
- **Chapitre 2** : Description de la zone d'étude et des données exploitées.
- **Chapitre 3** : Détail de la modélisation et du développement du système.
- **Chapitre 4** : Discussion des résultats obtenus, limites rencontrées et propositions d'amélioration.
- **Conclusion générale** : Synthèse des apports du projet et perspectives futures.

CADRE THÉORIQUE ET TECHNIQUE

Introduction

Ce chapitre présente les fondements théoriques et techniques sur lesquels repose le projet. Il aborde les concepts liés aux incendies de forêt, les Systèmes d'Information Géographique (SIG), la conversion de données spatiales, le web mapping avec TerriaJS, ainsi que l'application de la data science aux données géospatiales. Enfin, il décrit les outils et technologies utilisés.

1.1 Incendies de forêt : contexte et enjeux

1.1.1 Définition des incendies de forêt

Un *incendie de forêt*, également appelé feu de végétation, désigne une combustion non maîtrisée qui se propage dans des milieux naturels, affectant des zones boisées, des broussailles, des prairies ou des landes. Ce phénomène peut avoir des causes **naturelles** (comme la foudre) ou **anthropiques**, qu'elles soient *volontaires* (pyromanie) ou *accidentelles* (négligences humaines, activités agricoles ou industrielles).

La survenue d'un incendie repose sur la réunion de trois éléments fondamentaux, communément appelés le **triangle du feu** :

- une **source de chaleur** (étincelle, flamme, température élevée),
- un **combustible** (végétation sèche, bois mort, feuilles),
- de l'**oxygène** (présent naturellement dans l'atmosphère).

La propagation du feu dépend de plusieurs facteurs : la nature et la densité de la végétation, la topographie, les conditions météorologiques (température, humidité, vitesse du vent), ainsi que l'accessibilité du terrain. Les incendies peuvent entraîner des conséquences graves, tant sur le plan écologique (destruction d'écosystèmes, perte de biodiversité) qu'économique et humain (dommages matériels, menaces pour les populations).

1.1.2 Enjeux écologiques, économiques et sociaux

Les impacts des incendies sont multiples :

- **Écologiques** : destruction d'habitats, pollution de l'air, érosion des sols.
- **Économiques** : pertes agricoles, destruction d'infrastructures, coûts de la lutte anti-incendie.
- **Sociaux** : évacuations, perte de logements, stress pour les populations locales.

1.1.3 Exemples récents

Des incendies majeurs comme ceux de l’Amazonie (2019), de l’Australie (2019-2020), de la Gironde en France (2022), ou plus récemment de la Californie (2025) montrent l’ampleur croissante du phénomène et la nécessité de systèmes d’analyse et de prévention performants.

1.2 Systèmes d’Information Géographique (SIG)

Les Systèmes d’Information Géographique (SIG) sont des outils permettant de capturer, stocker, analyser et visualiser des données spatialisées. Ils jouent un rôle central dans la compréhension, la prévention et la gestion des incendies de forêt.

1.2.1 Types de données

- **Vecteur** : représentations géométriques (points, lignes, polygones).
- **Raster** : grilles de pixels représentant des valeurs continues (température, altitude, NDVI, etc.).

1.2.2 Applications des SIG dans les incendies

- Cartographie des zones brûlées.
- Suivi en temps réel via satellites.
- Analyse des zones à risque.
- Planification des interventions et accès.

1.3 Web mapping et TerriaJS

Le *web mapping* désigne l’ensemble des techniques permettant de visualiser et d’interagir avec des données cartographiques directement dans un navigateur web. Il repose sur des bibliothèques ou plateformes capables d’afficher des données géospatiales dynamiquement, souvent via des couches superposables issues de sources raster ou vecteur.

Dans le cadre de ce projet, deux outils complémentaires ont été mobilisés pour la cartographie web :

- **Leaflet (intégré dans React)** : utilisé dans l’interface *Dashboard* pour afficher dynamiquement les couches raster importées par l’utilisateur (fichiers `.tiff`). Cette bibliothèque JavaScript légère permet une interaction fluide avec la carte, notamment le zoom, le déplacement, et le rendu de couches raster via des extensions comme `georaster-layer-for-leaflet`.

- **TerriaJS** : framework de web mapping avancé, utilisé ici pour visualiser les couches **vecteur statiques** représentant les zones d’incendie, les routes, les limites administratives ou les types de végétation. TerriaJS offre une interface interactive puissante pour la superposition de couches et la navigation dans des jeux de données spatiaux.

L’association de Leaflet (pour les données raster dynamiques et locales dans React) et de TerriaJS (pour les données vectorielles statiques, centralisées sur le web) permet une couverture fonctionnelle large pour la visualisation spatiale.

1.4 Data science appliquée aux données spatiales

1.4.1 Notions de base

La data science regroupe des techniques d’analyse, de modélisation et de prédiction à partir de données. Appliquée aux données spatiales, elle permet :

- la **détection de tendances**,
- l’**identification de zones à risque**,
- la **prédiction de comportements** (propagation, saisonnalité...).

1.4.2 Exemples d’analyse

- **Clustering** des zones les plus brûlées,
- **Régression** pour modéliser la fréquence des feux,
- **Cartes des risques** de nouveaux départs de feu.

1.5 Outils et technologies utilisés

1.5.1 Les Environnements Matériels

Dans cette partie, nous présenterons l’environnement matériel et logiciel relatif à la réalisation de l’application. Pour réaliser notre travail, nous avons utilisé comme environnement matériel :

1.5.2 Outils et technologies utilisés

Le développement de l’application de visualisation et de gestion de données spatiales a nécessité le recours à un ensemble cohérent d’outils, bibliothèques et plateformes répartis entre le backend, le frontend, les environnements de développement, ainsi que des outils spécialisés dans le traitement de données géospatiales. Le choix de ces technologies repose sur leur robustesse, leur communauté active et leur compatibilité avec les standards du web et des SIG.

Machines	Caractéristiques
HP Probook	Système d'exploitation : Windows 11 Processeur : Intel Core I7 8ème génération Mémoire : 16 Go Disque Dur : 512 HDD
Ideapad Gaming 3	Système d'exploitation : Windows 11 Processeur : Intel Core I5 11ème génération Mémoire : 16 Go Disque Dur : 512 SSD

TABLEAU 1 : Environnement Matériel

1. Technologies backend

-  **Laravel (PHP)** : Framework PHP basé sur le modèle MVC, conçu pour le développement d'applications web robustes et sécurisées.
-  **Laravel Passport** : Package d'authentification OAuth2 complet pour Laravel, assurant la sécurité des API via les tokens.
-  **Python** : Langage polyvalent, utilisé pour le traitement des données et l'automatisation.
-  **Flask** : Micro-framework web léger en Python, utilisé pour les services API.
-  **PostgreSQL** : Système de gestion de base de données relationnelle open source.
-  **PostGIS** : Extension spatiale de PostgreSQL pour la gestion de données géographiques.
-  **Composer** : Gestionnaire de dépendances PHP utilisé avec Laravel.

2. Technologies frontend

-  **React** : Bibliothèque JavaScript développée par Meta pour les interfaces dynamiques.
-  **TerriaJS** : Framework pour la visualisation de données spatiales interactives.
-  **Leaflet** : Bibliothèque JS légère pour les cartes interactives.
-  **Axios** : Librairie JS pour les requêtes HTTP asynchrones.
-  **Bootstrap** : Framework CSS pour les interfaces responsive.

3. Environnement de développement

-  **Visual Studio Code** : Éditeur de code multiplateforme extensible pour le backend et frontend.
-  **XAMPP** : Environnement local de développement regroupant Apache, PHP, MariaDB.
-  **Git / GitHub** : Contrôle de version et collaboration via GitHub.
-  **Postman** : Outil de test et documentation d'API REST.
-  **Google Colab** : Notebook interactif basé sur Jupyter pour le traitement Python dans le cloud.
-  **Drawio** : Outil de création de diagrammes UML, architecture, etc.

4. Outils SIG et traitement de données spatiales

-  **QGIS** : SIG open source pour la visualisation, manipulation et conversion de données spatiales.

Conclusion

Ce chapitre a exposé les concepts théoriques et les technologies nécessaires à la compréhension et à la réalisation du projet. Il a permis d'identifier les outils appropriés à l'analyse, à la visualisation et à l'exploitation des données spatiales. Le chapitre suivant se consacrera à la description de la zone d'étude ainsi qu'aux données collectées et traitées.

ZONE D'ÉTUDE ET DONNÉES

Plan

1	Incendies de forêt : contexte et enjeux	3
2	Systèmes d'Information Géographique (SIG)	4
3	Web mapping et TerriaJS	4
4	Data science appliquée aux données spatiales	5
5	Outils et technologies utilisés	5

Introduction

Dans ce chapitre, nous présentons la zone d'étude ciblée par notre application, à savoir la région de la Kroumirie, connue pour sa richesse forestière et sa vulnérabilité aux incendies. Nous décrivons les différentes données spatiales collectées, en distinguant les couches vecteurs et rasters. Le chapitre détaille également les étapes de conversion, de pré-traitement ainsi que quelques exemples de cartes générées. Toutes les données spatiales ont été harmonisées au système de coordonnées de référence EPSG :32632 (WGS 84 / UTM zone 32N) pour garantir une cohérence géospatiale uniforme dans l'analyse et la visualisation.

2.1 Présentation de la Kroumirie

La Kroumirie est une région montagneuse située au nord-ouest de la Tunisie, s'étendant principalement dans le gouvernorat de Jendouba. Cette zone géographique se caractérise par un relief accidenté avec des altitudes variant entre 200 et 1200 mètres, et constitue l'une des régions les plus arrosées du pays avec des précipitations annuelles pouvant atteindre 1500 mm.

2.1.1 Caractéristiques physiques et écologiques

Le climat de la Kroumirie est de type méditerranéen subhumide à humide, caractérisé par des hivers doux et pluvieux et des étés chauds et secs. Cette particularité climatique favorise le développement d'une végétation dense et variée.

La région abrite les forêts les plus importantes de Tunisie, principalement composées de :

- Chêne-liège (*Quercus suber*) sur les versants bien exposés
- Chêne zéen (*Quercus canariensis*) dans les vallons humides
- Pin maritime (*Pinus pinaster*) et pin d'Alep (*Pinus halepensis*) dans les zones reboisées
- Maquis méditerranéen composé d'arbousiers, bruyères, myrtes et cistes

Le réseau hydrographique est dense, avec de nombreux oueds et cours d'eau temporaires qui s'activent principalement pendant la saison des pluies (octobre à mars).

2.1.2 Contexte socio-économique

La Kroumirie est une région à vocation essentiellement forestière et agro-pastorale. Elle est caractérisée par :

- Une population rurale relativement dense vivant en habitat dispersé (douars)

- Une économie locale fortement dépendante des ressources forestières (liège, bois de chauffage, plantes aromatiques et médicinales)
- Des activités agricoles traditionnelles (élevage extensif, cultures céréalières, arboriculture)
- Un taux de pauvreté supérieur à la moyenne nationale

2.1.3 Vulnérabilité aux incendies

Plusieurs facteurs font de la Kroumirie une zone particulièrement vulnérable aux incendies de forêt :

- La densité et la continuité du couvert végétal
- L'accumulation de biomasse combustible au sol (litière)
- Le relief accidenté qui complique l'accès aux zones sensibles
- Les périodes de sécheresse estivale de plus en plus marquées
- Les pratiques agricoles traditionnelles (écobuage)
- La pression anthropique (exploitation forestière, tourisme)

Cette vulnérabilité justifie la mise en place d'outils de surveillance et de prévention adaptés, comme le système d'information géographique développé dans le cadre de ce projet.

2.2 Données spatiales collectées

La diversité des facteurs influençant le déclenchement et la propagation des incendies nécessite la collecte et l'intégration de données spatiales variées. Ces données ont été obtenues auprès de différentes institutions nationales et internationales.

2.2.1 Cartes vecteurs initiales

Les shapefiles, contrôlés techniquement, ont été reprojetés en EPSG :32632 :

- Limites administratives ;
- Réseau routier
- Séries forestières
- Parcelles agricoles
- Hydrographie
- Kroumirie
- Infrastructures hydrauliques

- Postes de vigie
- Villages

Ces données vectorielles ont été acquises via des plateformes de partage institutionnelles et ont fait l'objet d'un contrôle qualité préalable à leur intégration dans le système (vérification de la topologie, des attributs, et mise à jour si nécessaire).

Les figures suivantes présentent quelques exemples de couches vectorielles qui ont été reprojetées dans le système de coordonnées EPSG :32632 (WGS 84/UTM zone 32N) afin d'assurer leur parfait alignement avec les autres données spatiales utilisées dans cette étude :

La figure 1 présente la répartition des séries forestières dans la région de Jendouba (Kroumirie).

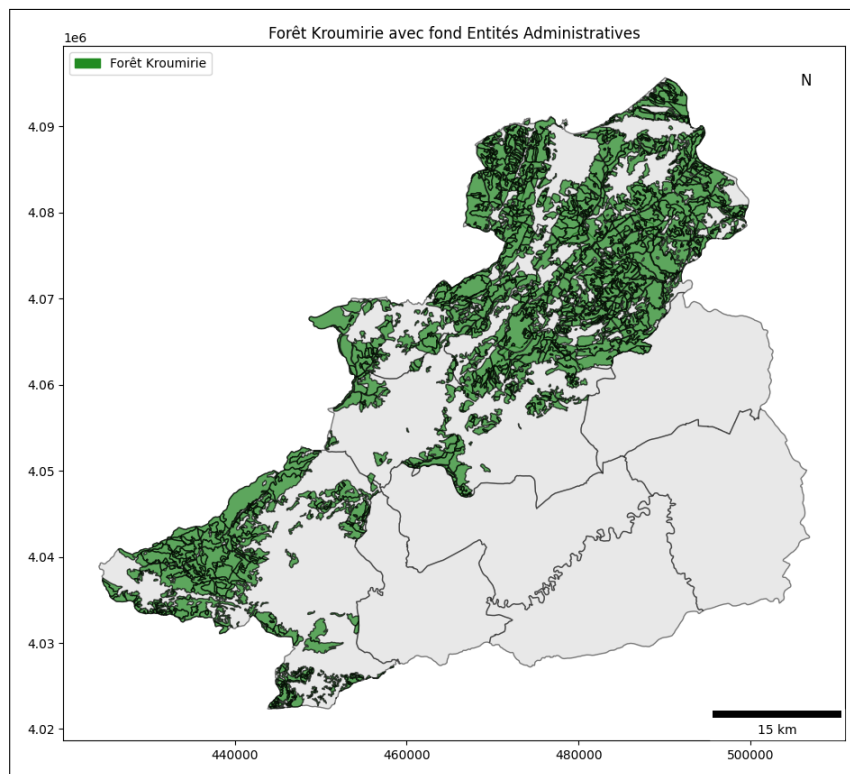


FIGURE 1 : Carte des séries forestières de la Kroumirie (gouvernorat de Jendouba, polygones)

Cette figure 2 représente une couche vecteur illustrant les séries forestières de Jendouba.

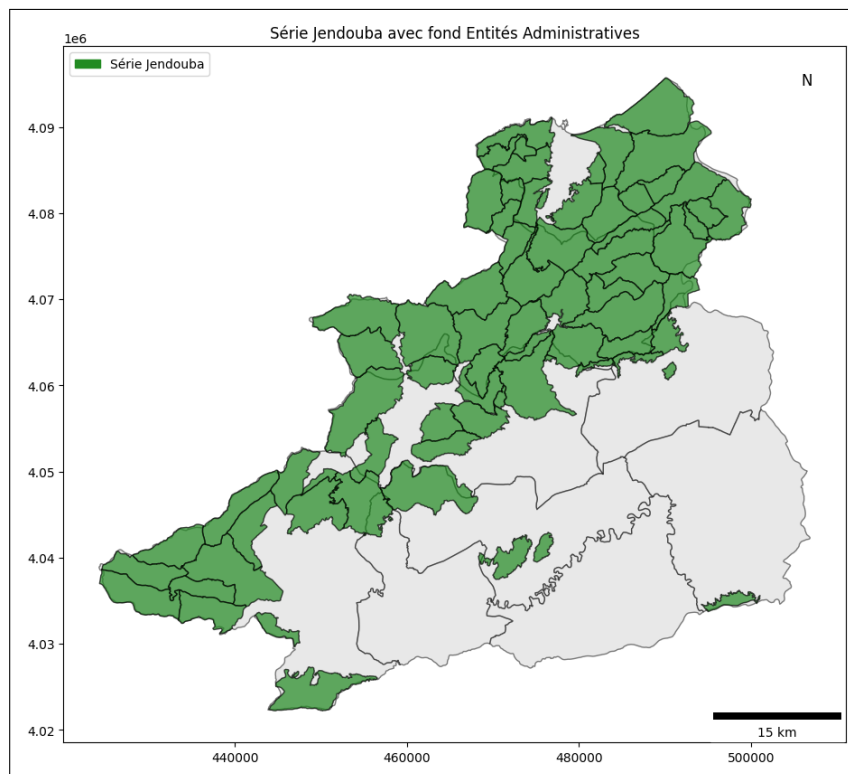


FIGURE 2 : Couche vecteur : Séries forestières de Jendouba(polygones)

Cette figure 3 représente une couche vecteur des parcelles agricoles.

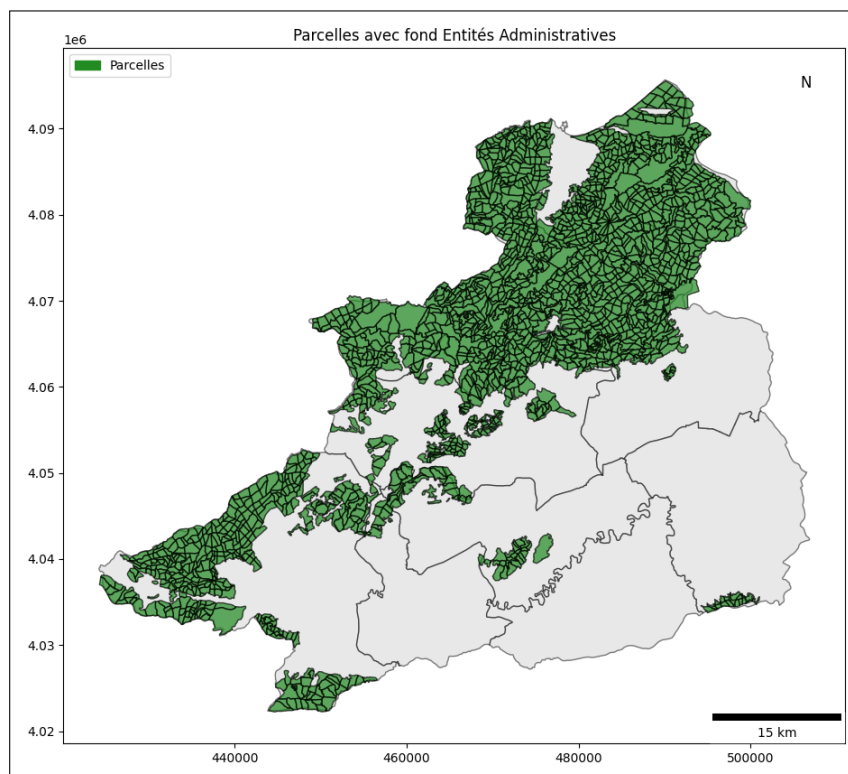


FIGURE 3 : Couche vecteur : Parcelles forestières de Jendouba(polygones)

Cette figure 4 représente une couche SIG vecteur illustrant les routes.

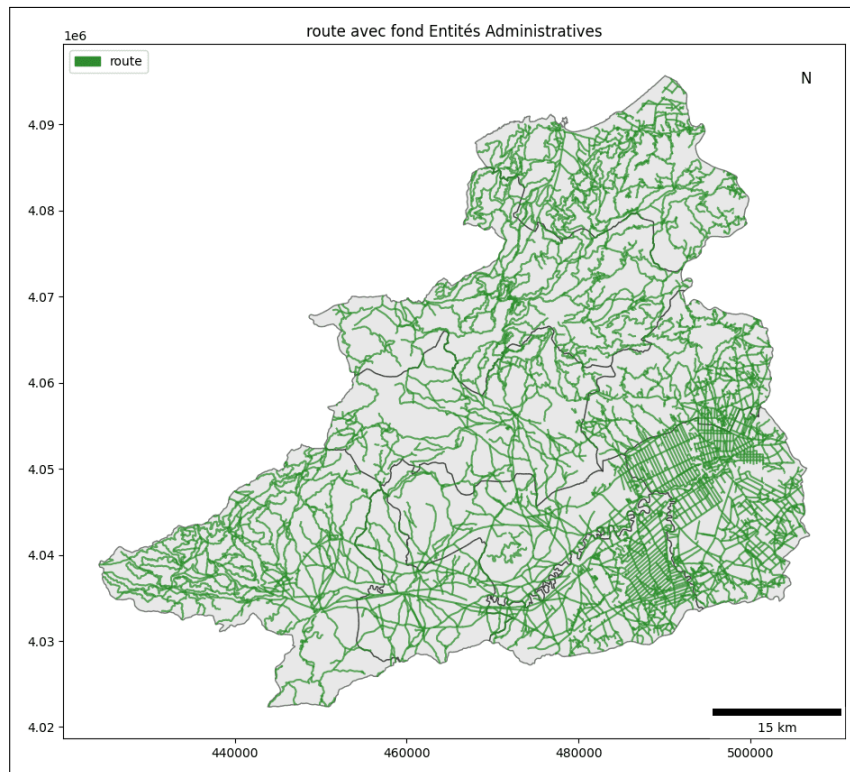


FIGURE 4 : Couche vecteur : Réseau routier(lignes)

Cette figure 5 représente une couche vecteur des barrages collinaires ponctuels.

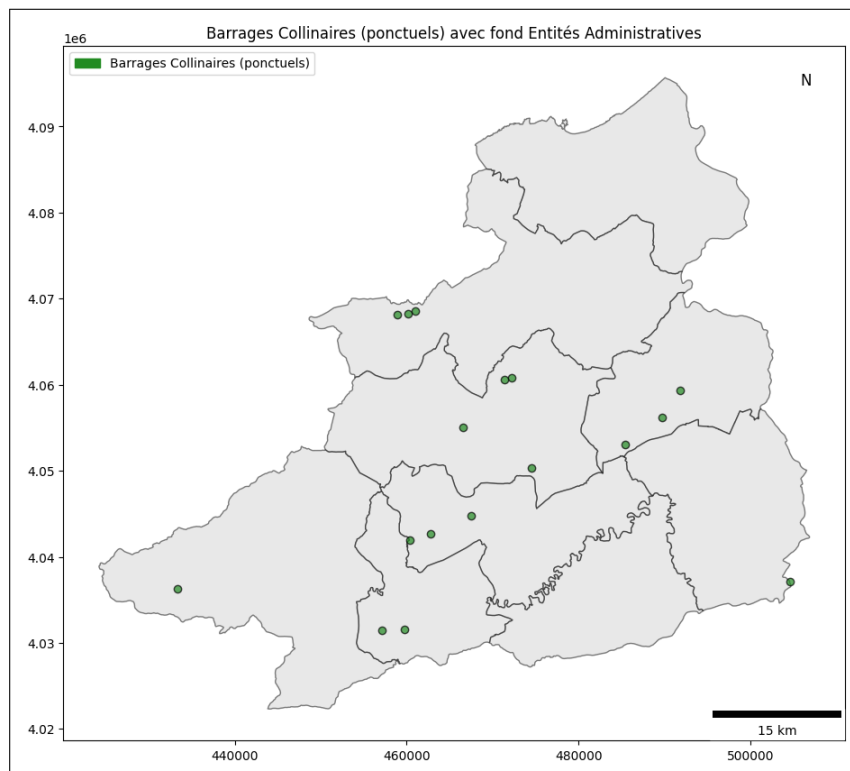


FIGURE 5 : Couche vecteur : Barrages collinaires (points)

Cette figure 6 représente une couche vecteur des Villages.

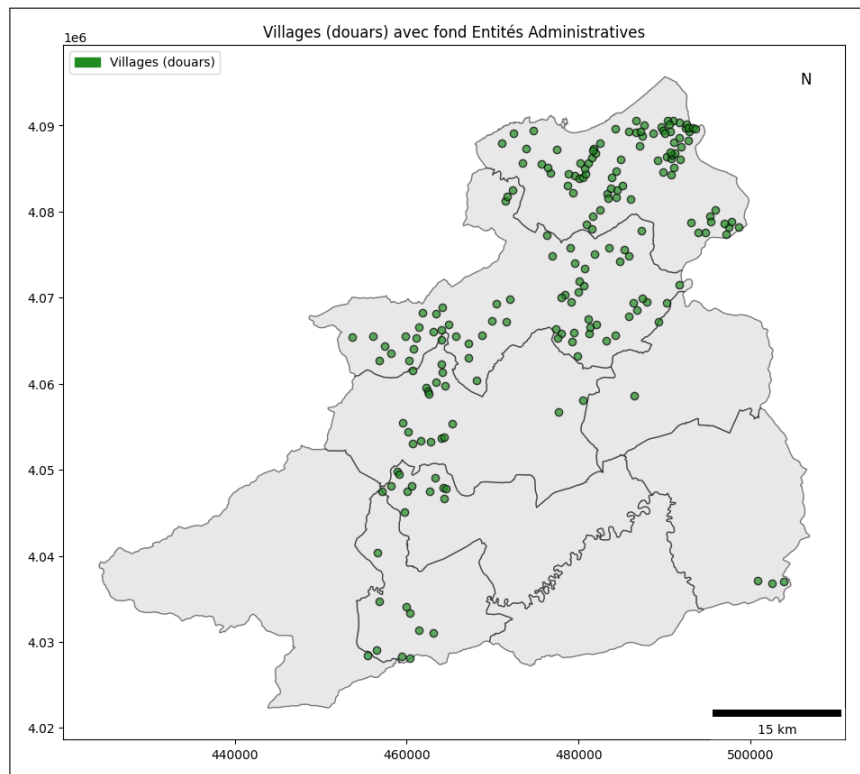


FIGURE 6 : Couche vecteur : Villages (points)

2.2.2 Cartes raster existantes

En complément des données vectorielles, plusieurs couches raster ont été intégrées au système pour permettre une analyse plus fine des facteurs environnementaux. Les principales couches raster utilisées sont :

- **Modèle numérique de terrain (MNT)** : Utilisé pour analyser l'altitude, la pente et l'exposition du relief.
- **Occupation du sol et couverture forestière** :
 - Forêt de la Kroumirie
 - Zones urbaines (terrains construits)
 - Périmètres irrigués
 - Unités de transformation agricole
- **Hydrologie et infrastructures hydrauliques** :
 - Infrastructures hydrauliques surfaciques (grands barrages)
 - Barrages collinaires
 - Lacs collinaires
 - Hydrographie linéaire

— **Accessibilité et réseaux :**

- Réseau routier (routes)

— **Analyse de proximité :**

- Proximité des infrastructures hydrauliques linéaires
- Proximité des infrastructures hydrauliques ponctuelles (barrages collinaires, forages d'eau)
- Proximité des objets hydrographiques isolés
- Proximité des postes vigies
- Proximité des tranchées pare-feu
- Proximité des villages/douars

Les figures suivantes présentent un ensemble de couches raster reprojetées dans le système de coordonnées EPSG :32632 (WGS 84/UTM zone 32N), garantissant une parfaite cohérence spatiale avec les autres données géographiques utilisées dans cette étude :

Cette figure 7 représente une couche raster des barrages collinaires ponctuels.

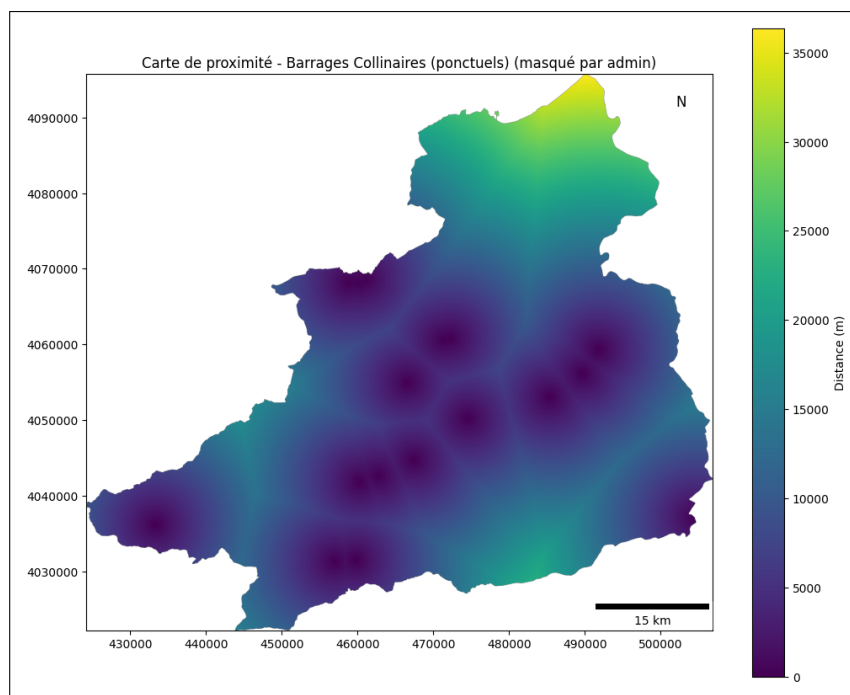


FIGURE 7 : Couche raster : Barrages collinaires (ponctuels)

Cette figure 8 représente une couche raster de l'hydrographie linéaire.

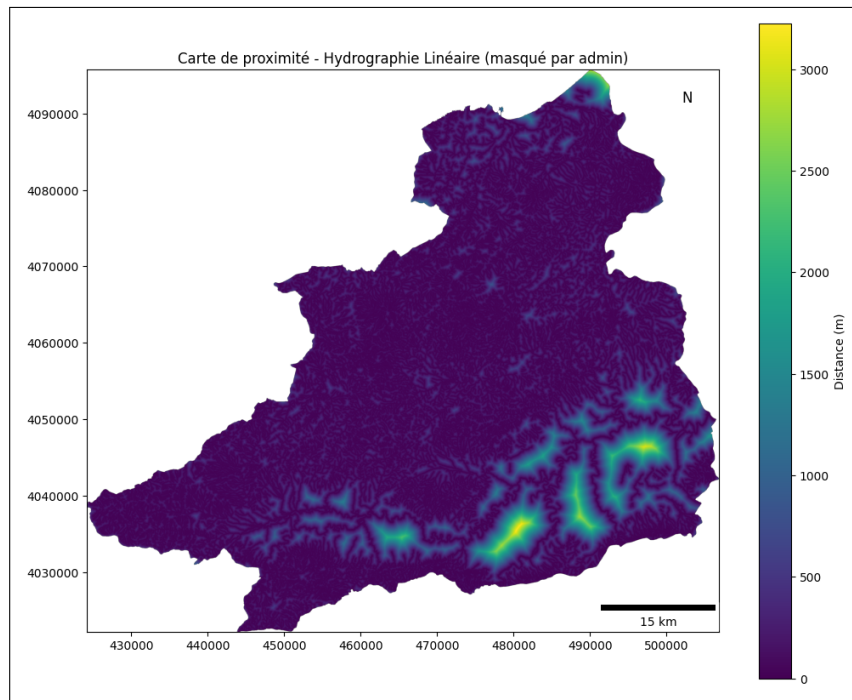


FIGURE 8 : Couche raster : Hydrographie linéaire

Cette figure 9 représente une couche raster des infrastructures hydrauliques ponctuelles (lacs collinaires).

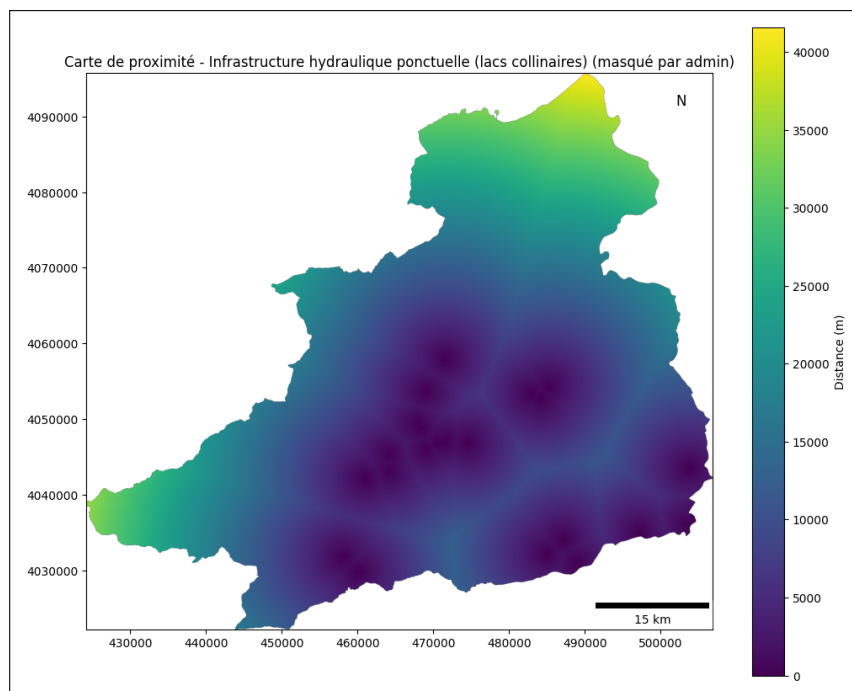


FIGURE 9 : Couche raster : Infrastructure hydraulique ponctuelle – lacs collinaires

Cette figure 10 représente une couche raster des grands barrages.

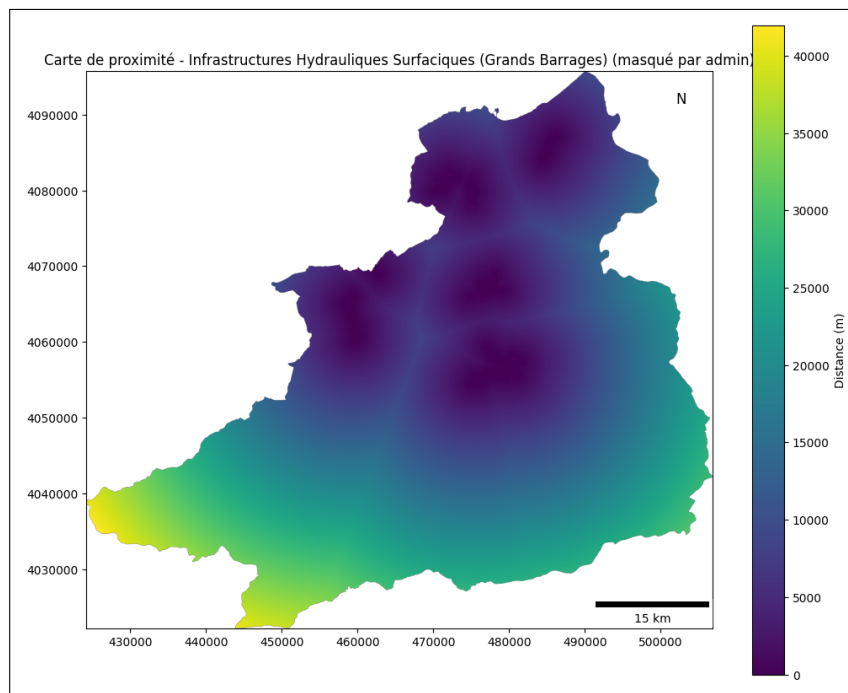


FIGURE 10 : Couche raster : Infrastructures Hydrauliques Surfiques – Grands barrages

Cette figure 11 représente une couche raster des infrastructures hydrauliques surfiques.

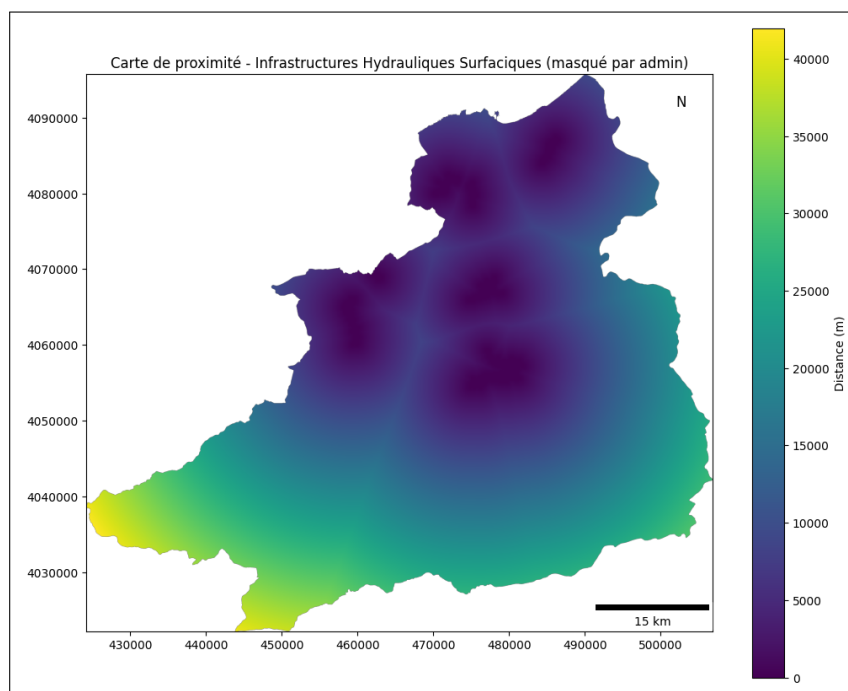


FIGURE 11 : Couche raster : Infrastructures Hydrauliques Surfiques

Cette figure 12 représente une couche raster des lacs collinaires.

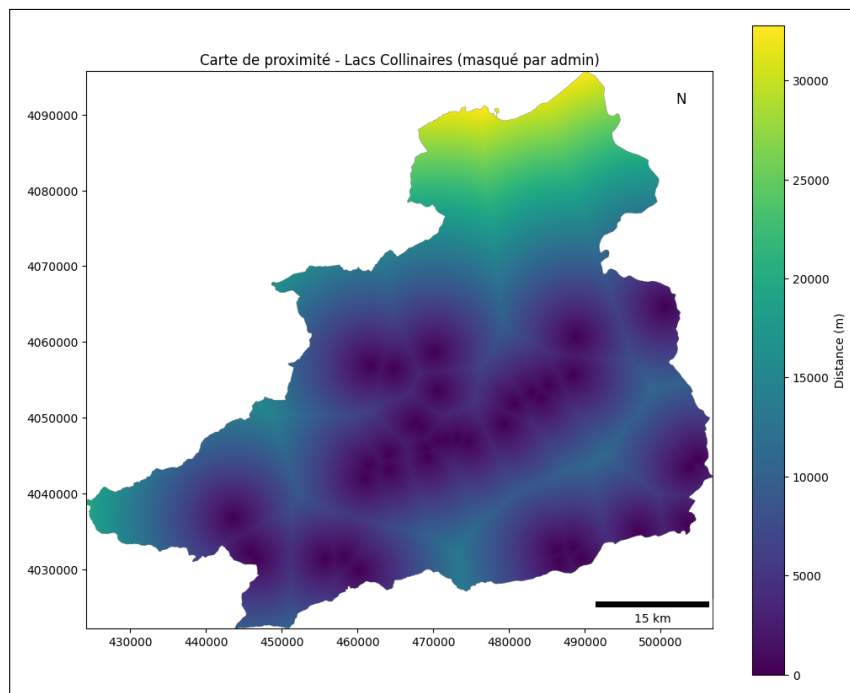


FIGURE 12 : Couche raster : Lacs collinaires

Cette figure 13 représente une couche raster des périmètres irrigués.

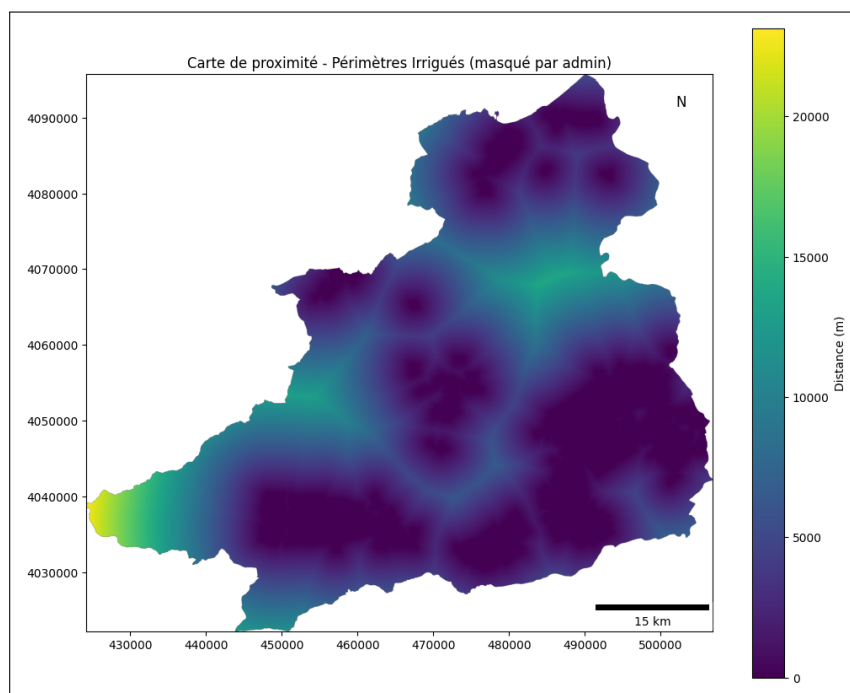


FIGURE 13 : Couche raster : Périmètres irrigués

Cette figure 14 représente une couche raster du réseau routier.

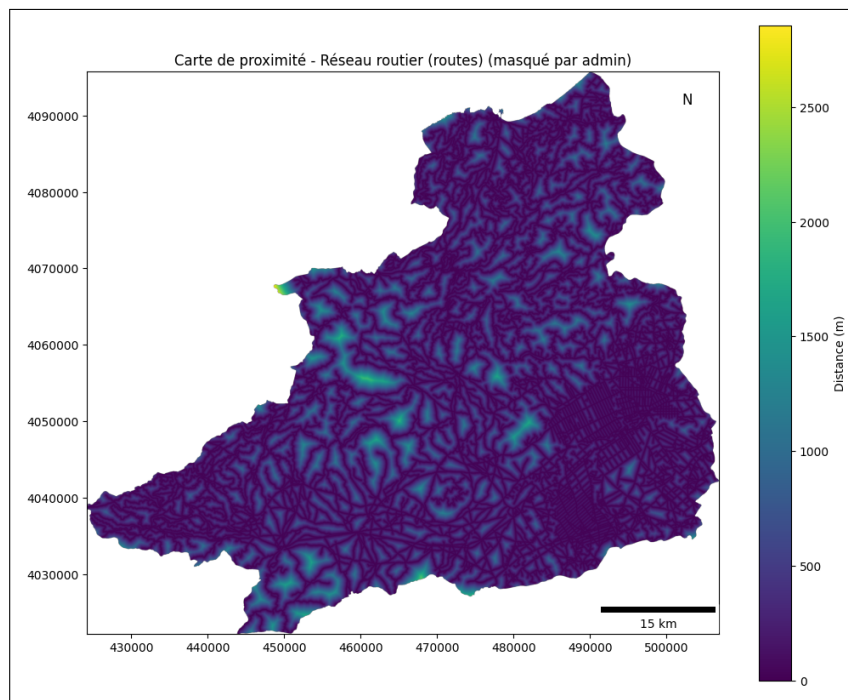


FIGURE 14 : Couche raster : Réseau routier

Cette figure 15 représente une couche raster des unités de transformation agricole.

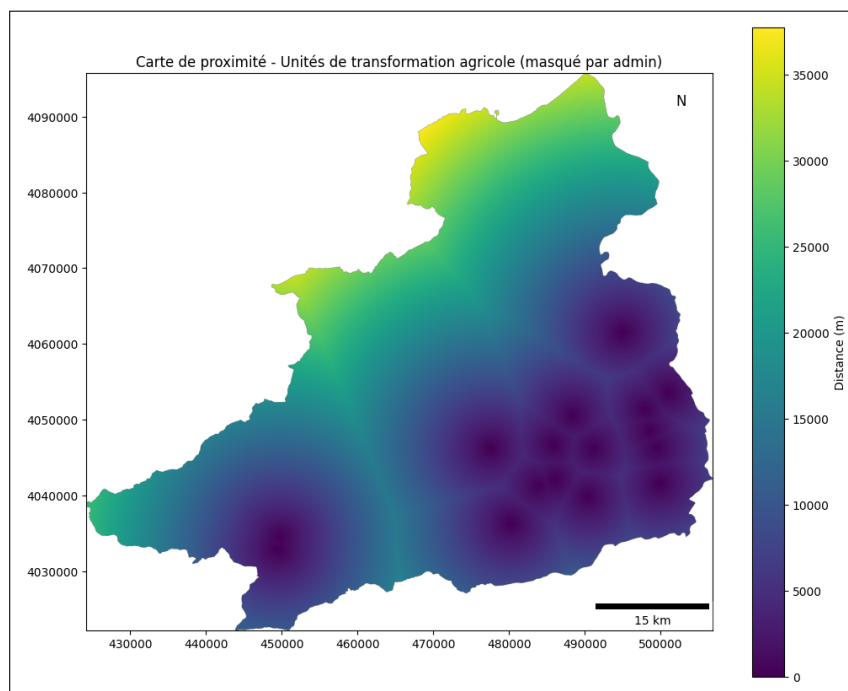


FIGURE 15 : Couche raster : Unités de transformation agricole

Cette figure 16 représente une couche raster des zones urbaines construites.

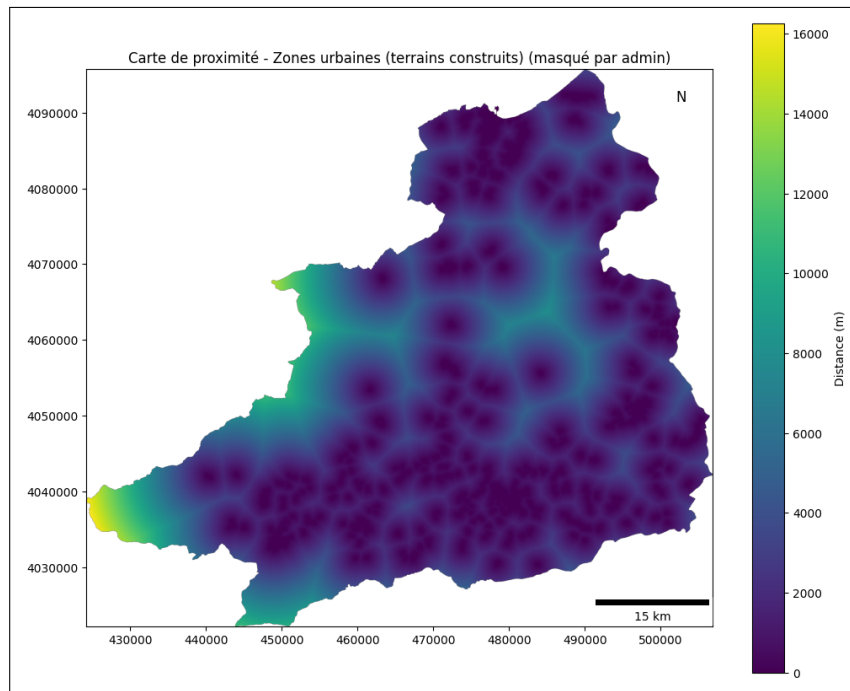


FIGURE 16 : Couche raster : Zones urbaines (terrains construits)

2.3 Conversion des vecteurs en rasters

2.3.1 Intérêt de la conversion

La conversion des données vecteur en raster présente plusieurs avantages dans le cadre de l'analyse spatiale :

- Simplification des calculs spatiaux tels que les distances (notamment la distance euclidienne), le zonage ou l'interpolation ;
- Application d'algorithmes avancés de traitement d'image, notamment grâce à un script Python développé pour extraire les valeurs statiques des incendies à partir des couches raster, en fonction de leur localisation ;
- Croisement facilité avec d'autres couches raster (occupation du sol, altitude, température, etc.).

2.3.2 Méthodologie

Deux approches ont été utilisées pour convertir les entités vecteur en raster :

2.3.2.1 Méthode 1 – Outils de QGIS

- Rasterisation via l'outil "*Vecteur vers raster*" de GDAL ;
- Calcul des distances à l'aide de "*Proximité (raster distance)*" ;

- Méthode simple mais manuelle, parfois instable car dépendante du choix arbitraire de la résolution du raster (taille des pixels), ce qui peut nuire à la qualité des résultats.

2.3.2.2 Méthode 2 – Script Python automatisé

- Chargement des couches vecteur avec `geopandas` ;
- Définition de l'emprise et de la résolution du raster avec `rasterio.transform.from_bounds` ;
- Rasterisation binaire à l'aide de `rasterio.features.rasterize` ;
- Calcul des distances (pixels) avec `scipy.ndimage.distance_transform_edt` ;
- Masquage spatial des rasters selon les limites administratives via `rasterio.mask` ;
- Export des rasters au format GeoTIFF et visualisation avec `matplotlib`.

2.4 Données d'incendies

Les incendies de forêt sont en augmentation dans de nombreuses régions, tant en fréquence qu'en intensité. En Europe, des pays comme l'Espagne, le Portugal, la France et la Grèce enregistrent des milliers d'hectares brûlés chaque année. L'évolution du climat, conjuguée aux pratiques humaines, joue un rôle central dans cette tendance.

2.4.1 Sources des données d'incendies

Dans le cadre de ce projet, nous avons étudié les incendies survenus dans la région de Jendouba entre 2000 et 2024. Les données utilisées proviennent de trois sources principales :

- **EFFIS** (European Forest Fire Information System), fournissant des données européennes harmonisées,
- **NASA FIRMS** (Fire Information for Resource Management System), avec des détections par satellite à haute fréquence,
- **DGF** (Direction Générale des Forêts), offrant des statistiques locales détaillées.

Source	Période couverte	Nombre total d'incendies
EFFIS	2011–2024	138
NASA FIRMS	2001–2024	559
DGF	2000–2016	427

TABLEAU 2 : Comparaison des données d'incendies pour la région de Jendouba selon trois sources

La figure 17 illustre la répartition géographique des incendies issus de différentes sources dans la région de Jendouba entre 2000 et 2024.

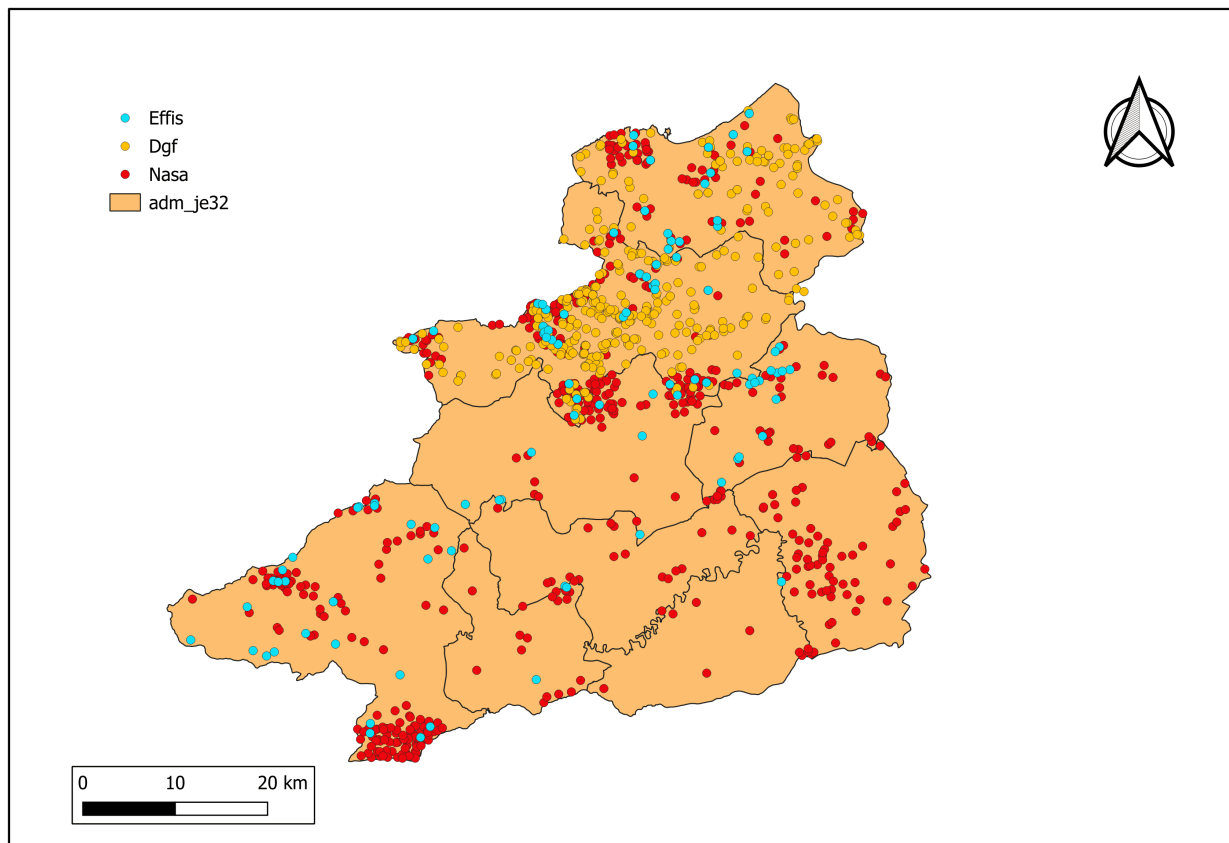


FIGURE 17 : Répartition géographique des incendies dans la région de Jendouba (données 2000–2024)

2.4.2 Données météorologiques associées

Pour enrichir l'analyse, des données météorologiques ont été intégrées manuellement à chaque enregistrement d'incendie. Cela inclut :

- des **données annuelles** : *vitesse et direction du vent, précipitations, humidité moyenne, température moyenne, température maximale et minimale,*
- des **données journalières** spécifiques au jour de l'incendie : *température et humidité.*

Les sources de ces données météorologiques sont les suivantes :

- **Meteostat** (données journalières disponibles à partir de 2016),
- **Weather Spark** (données journalières historiques de 2000 à aujourd'hui),
- **NASA POWER** (pour certaines données atmosphériques complémentaires).

Cette combinaison de sources permet une analyse complète et comparative de l'évolution spatio-temporelle des incendies et de leur lien potentiel avec les conditions climatiques locales.

2.5 Pré-traitement des données raster

Le pré-traitement des données raster a impliqué plusieurs étapes afin d'assurer la précision et la pertinence des informations utilisées pour l'analyse. Ces étapes sont les suivantes :

- **Reprojection des données** : Les couches raster ont été projetées dans le système de coordonnées EPSG :32632, qui correspond à la zone d'étude de la Kroumirie.
- **Alignement des résolutions et des étendues spatiales** : Un ajustement a été effectué pour uniformiser les résolutions et les étendues géographiques des différentes couches raster, assurant ainsi leur compatibilité.
- **Découpage selon la zone de Jendouba** : Les données ont été découpées en fonction de la région de Jendouba, en utilisant les limites administratives de la région pour cibler l'analyse.
- **Traitement des valeurs manquantes** : Des méthodes d'interpolation et de filtrage ont été appliquées pour combler les valeurs manquantes, permettant de maintenir la qualité et la continuité des données.
- **Filtrage et lissage des données** : Des techniques de filtrage comme le filtre médian et le flou gaussien ont été utilisées pour lisser les données et améliorer leur qualité visuelle.

Conclusion

Ce chapitre a permis de définir le contexte géographique de la Kroumirie et de présenter les données spatiales essentielles au projet. La préparation de ces données garantit une base solide pour les analyses des incendies à venir. Elles serviront également de fondation pour l'analyse et la modélisation qui seront approfondies dans les chapitres suivants.

MODÉLISATION ET DÉVELOPPEMENT DU SYSTÈME

Plan

1	Présentation de la Kroumirie	9
2	Données spatiales collectées	10
3	Conversion des vecteurs en rasters	20
4	Données d'incendies	21
5	Pré-traitement des données raster	23

Introduction

Ce chapitre décrit en détail les choix méthodologiques, les étapes de conception et les implémentations techniques réalisées dans le cadre du projet. Il débute par la justification de l'approche en cascade, puis présente les différents diagrammes de modélisation, l'architecture globale du système, ainsi que les technologies choisies pour le développement backend et frontend. Ensuite, nous abordons l'intégration de la data science à travers l'extraction des valeurs raster, la modélisation prédictive et l'intégration des résultats dans l'application web.

3.1 Justification du modèle en cascade

Le choix du modèle en cascade pour le développement de cette application s'appuie sur la nature progressive et bien structurée du projet. Le modèle en cascade permet une séparation claire entre les phases de spécification, de conception, de développement et de tests, garantissant ainsi une meilleure traçabilité des exigences. Ce modèle est particulièrement adapté lorsque les besoins sont bien définis dès le départ, ce qui est le cas ici grâce à une analyse préalable rigoureuse. Il offre également l'avantage d'une documentation systématique à chaque étape, facilitant la maintenance et l'évolution future du système.

3.2 Diagrammes de modélisation

3.2.1 Diagramme des phases (cascade)

Le diagramme des phases met en évidence l'enchaînement logique des étapes du projet : analyse des besoins, conception, implémentation, tests et déploiement. Chaque phase est validée avant le passage à la suivante, assurant ainsi un développement contrôlé et cohérent.

3.2.2 Diagrammes UML

Pour formaliser la conception du système, plusieurs diagrammes UML ont été élaborés :

- **Diagramme de cas d'utilisation** : il présente les différents acteurs du système (forestier, ingénieur, visiteur, administrateur) et leurs interactions avec les principales fonctionnalités (visualisation de couches, gestion des utilisateurs, déclaration d'incendies, etc.).

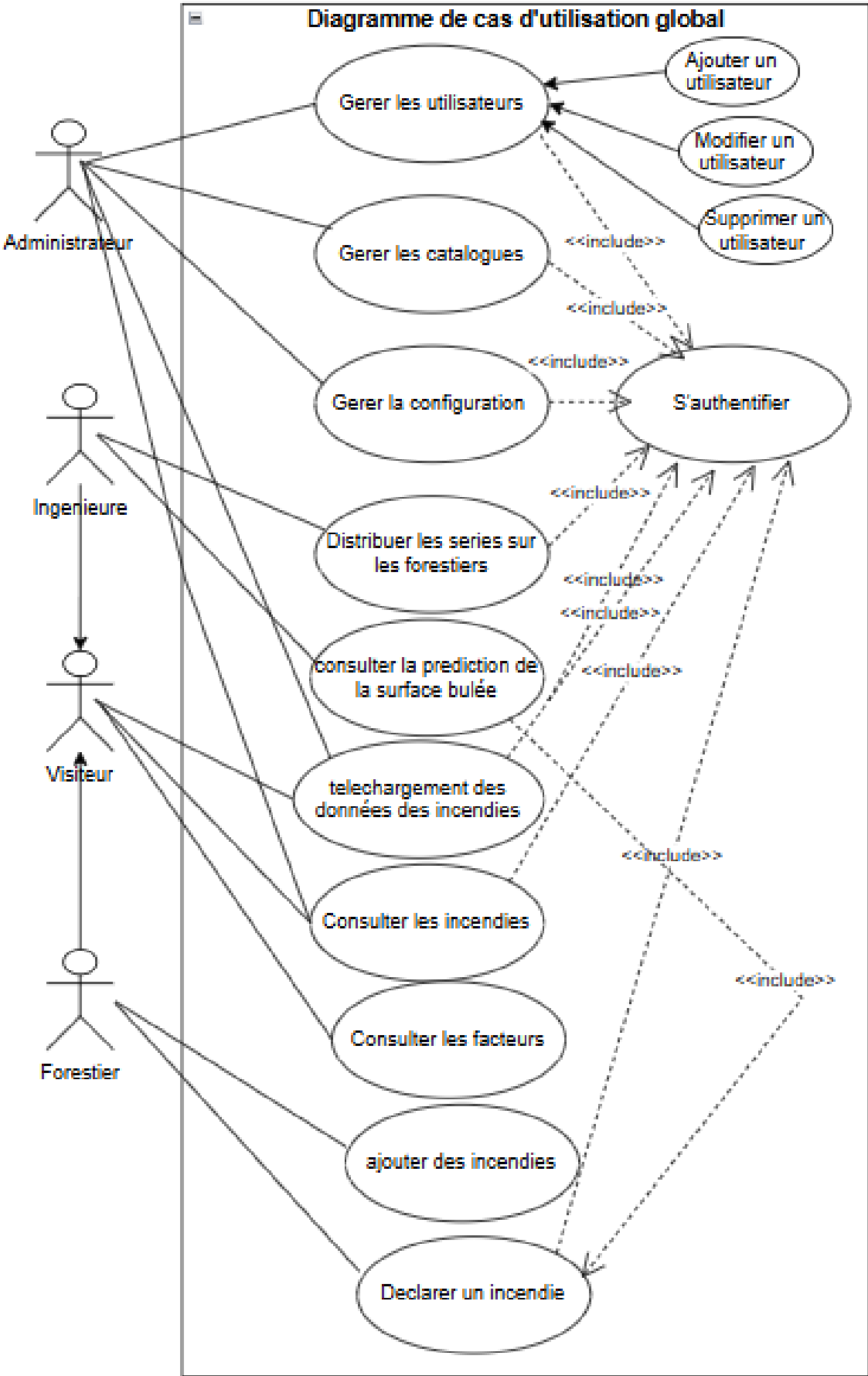


FIGURE 18 : Diagramme de cas d'utilisation

Descriptions textuelles des cas d'utilisation

Dans cette section, nous présentons une série de descriptions textuelles des cas d'utilisation du système. Ce type de diagramme est utilisé pour illustrer, de manière détaillée, les différentes interactions entre les utilisateurs (ou acteurs) et le système à travers des scénarios concrets. Il permet une meilleure compréhension des fonctionnalités et du comportement attendu du système.

Liste des cas d'utilisation présentés

- Authentification
- Déclarer un incendie
- Attribuer les séries forestières

Cas d'utilisation : Authentification

TABLEAU 3 : Description textuelle du cas d'utilisation « Authentification »

Cas d'utilisation :	Authentification
Résumé :	Ce cas d'utilisation permet à un utilisateur de s'authentifier auprès du système. Le système vérifie les identifiants saisis en consultant la base de données, puis autorise ou refuse l'accès selon que l'utilisateur est reconnu comme valide ou non.
Acteurs :	Utilisateur, Système, Base de données
Pré-conditions :	— L'utilisateur dispose d'un compte actif avec des identifiants valides (login et mot de passe). — Le système et la base de données sont opérationnels.

Scénario nominal :	1. L'utilisateur saisit ses identifiants dans le formulaire de connexion. 2. Le système reçoit les identifiants et initie une requête de vérification auprès de la base de données. 3. La base de données traite la demande de validation. 4. La base de données retourne le résultat de la validation au système. 5. Si la réponse indique que les identifiants sont valides (valide = True), le système confirme la connexion réussie à l'utilisateur.
Scénario alternatif :	— Si la base de données retourne valide = False, le système affiche un message de connexion erronée. — Si l'utilisateur saisit un identifiant ou mot de passe vide, le système refuse l'envoi et demande de compléter les champs. — En cas de problème de connexion au système ou à la base de données, une erreur technique est affichée. — Si plusieurs échecs de connexion sont détectés, un blocage temporaire du compte peut être mis en place.

Cas d'utilisation : Déclarer un incendie

TABLEAU 5 : Description textuelle du cas d'utilisation « Déclarer un incendie »

Cas d'utilisation :	Déclarer un incendie
Résumé :	Ce cas d'utilisation permet à un forestier de déclarer un incendie via une interface, déclenchant une série d'actions automatisées dans le système, notamment la notification par mail, la visualisation de l'incendie.
Acteurs :	Forestier

Pré-conditions :	— Le forestier s’est authentifié avec succès. — L’accès au module de déclaration est autorisé.
Scénario nominal :	1. Le forestier se connecte au système. 2. Le système effectue l’authentification. 3. Une fois authentifié, le forestier accède à l’option « Déclarer un incendie ». 4. Le système donne l’accès à la déclaration. 5. Le forestier déclenche la déclaration d’incendie. 6. Le système envoie automatiquement un mail de notification à l’ingénieur. 7. Le système permet la visualisation de l’incendie déclaré.
Scénario alternatif :	— Si l’authentification échoue, le forestier ne peut pas accéder à la déclaration. — Si l’adresse mail de l’ingénieur est invalide, il ne sera pas notifié.

Cas d’utilisation : Attribuer les séries forestières

TABLEAU 7 : Description textuelle du cas d’utilisation « Attribuer les séries forestières »

Cas d’utilisation :	Attribuer les séries forestières
Résumé :	Ce cas d’utilisation permet à un ingénieur de se connecter au système, consulter les séries forestières ainsi que les forestiers disponibles, et attribuer les séries aux forestiers.
Acteurs :	Ingénieur, Système

Pré-conditions :	— L'ingénieur dispose d'un compte valide. — L'accès au système est fonctionnel.
Scénario nominal :	1. L'ingénieur se connecte au système. 2. Le système authentifie l'ingénieur et retourne une confirmation. 3. L'ingénieur consulte la liste des séries forestières et des forestiers. 4. Le système retourne les données demandées. 5. L'ingénieur sélectionne et attribue les séries à un ou plusieurs forestiers. 6. Le système enregistre les attributions. 7. L'ingénieur se déconnecte du système.
Scénario alternatif :	— Si l'identifiant ou le mot de passe de l'ingénieur est incorrect, la connexion est refusée. — Si le système rencontre une erreur lors du chargement des données, un message d'indisponibilité est présenté. — Si une erreur survient lors de l'attribution des séries, un message est affiché et aucune modification n'est enregistrée. — Si un forestier est déjà assigné à une autre série incompatible, un message d'avertissement est affiché. — Si la connexion Internet est interrompue, le système empêche toute action. — Si l'ingénieur tente d'attribuer une série non disponible, une erreur est affichée.

— **Diagramme de classes** : il illustre la structure des entités principales telles que les utilisateurs,

les couches spatiales, les alertes d'incendie, et leurs relations.

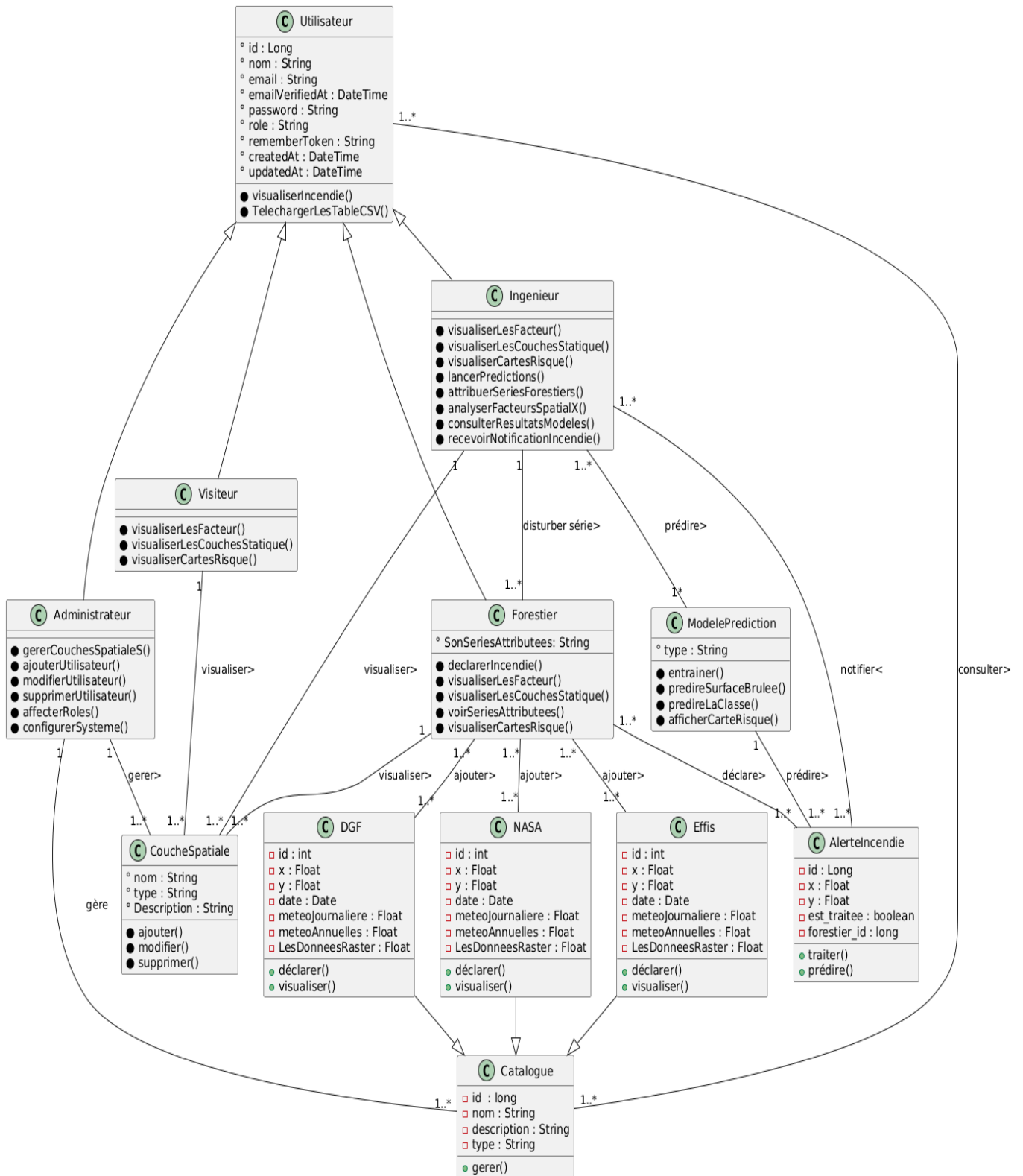


FIGURE 19 : Diagramme de classes du système

- **Diagrammes de séquence** : ils montrent le déroulement des interactions entre le frontend et le backend lors des opérations critiques. Chaque sous-diagramme correspond à un scénario fonctionnel précis :

- **Authentification** : décrit la procédure d'authentification d'un utilisateur à partir du formulaire de connexion.

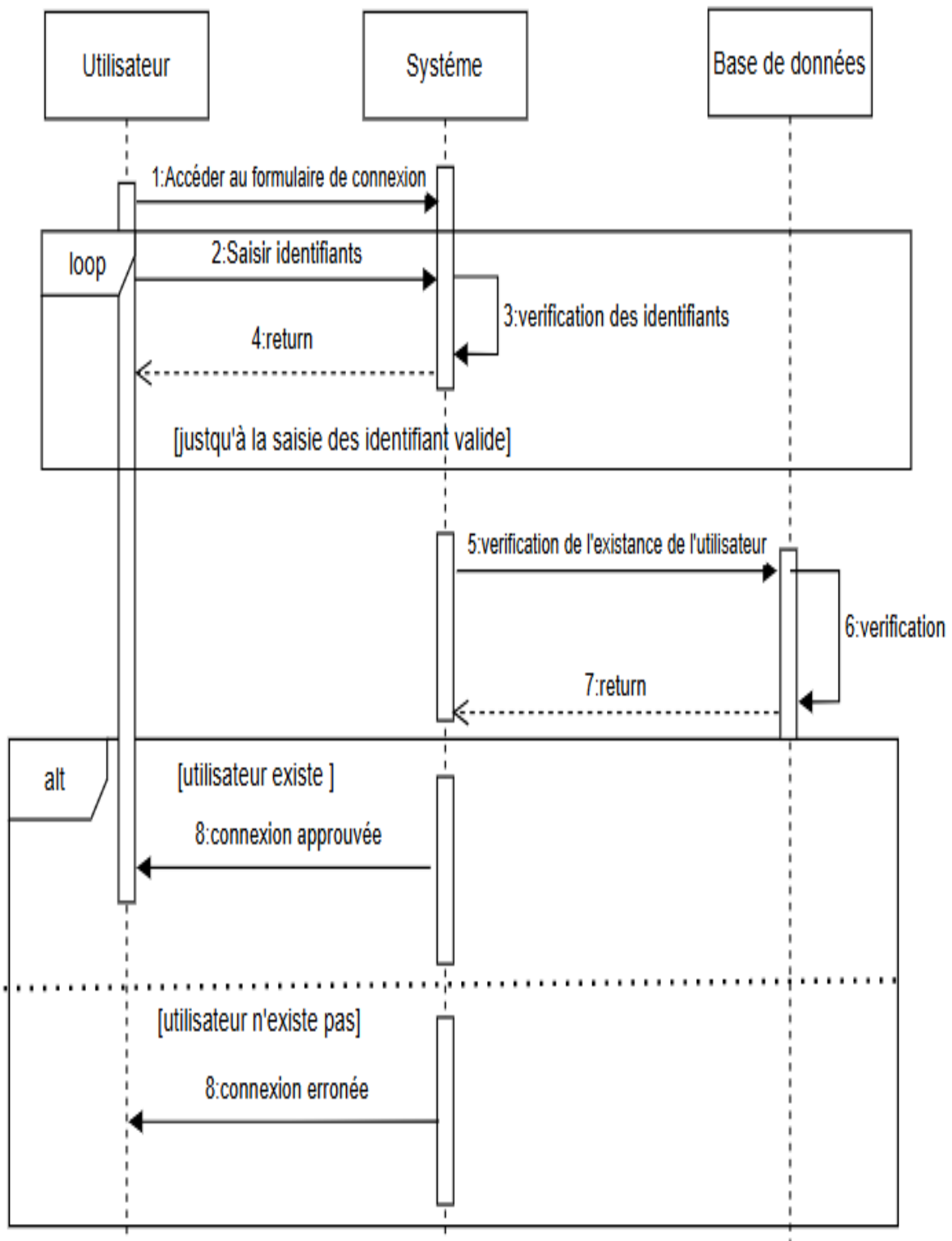


FIGURE 20 : Diagramme de séquence — Authentification

- **Création d'un utilisateur** : montre les échanges lors de l'ajout d'un nouvel utilisateur par l'administrateur.

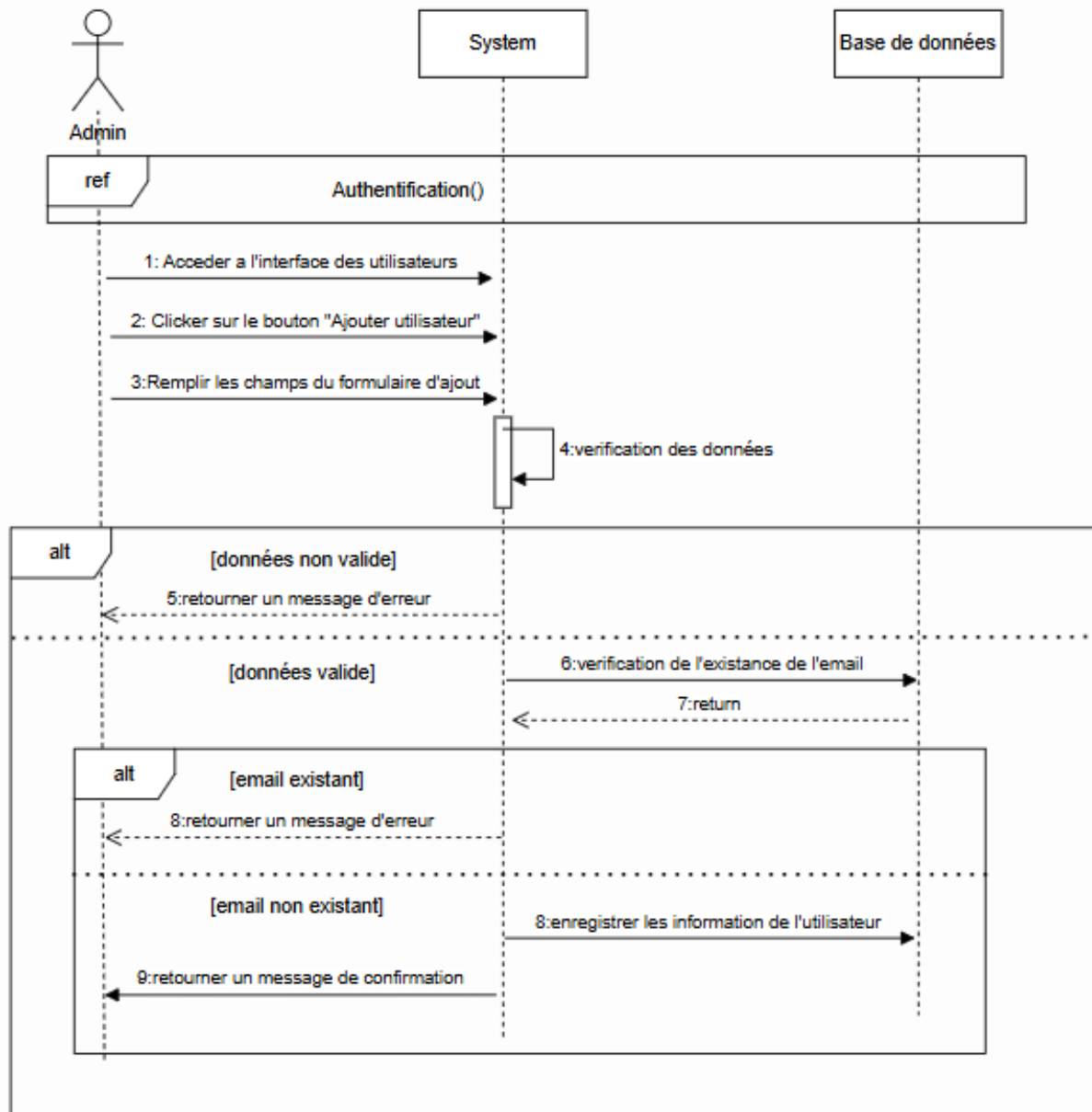


FIGURE 21 : Diagramme de séquence — Création d'un utilisateur

— **Distribution des séries** : illustre l'attribution d'une série forestière à un forestier.

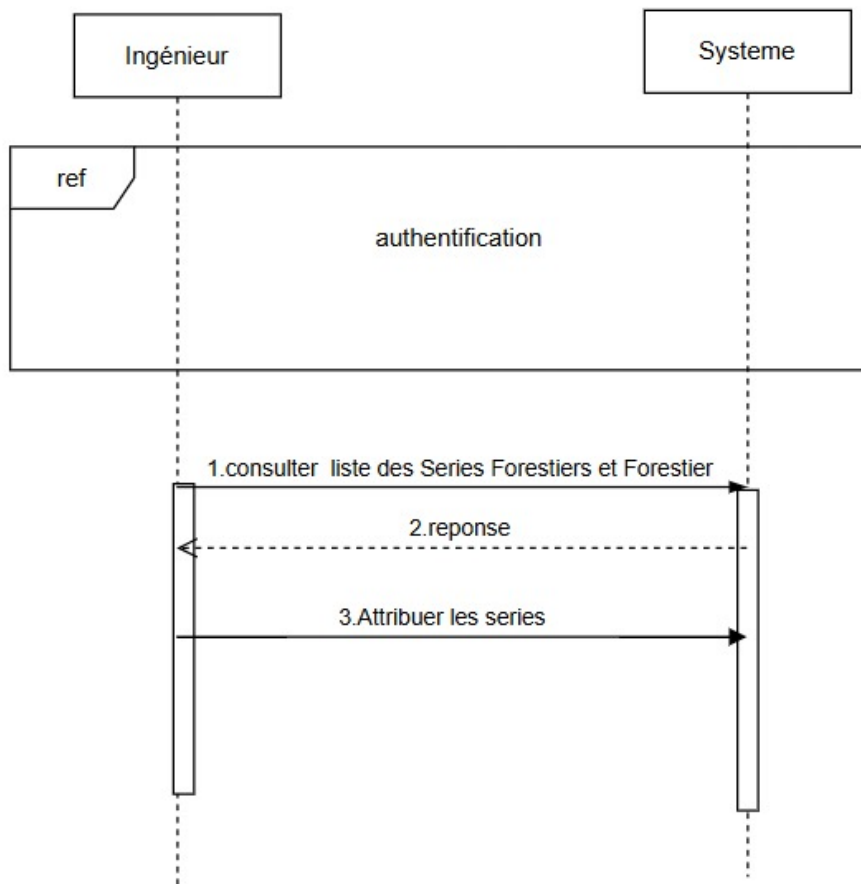


FIGURE 22 : Diagramme de séquence — Distribution d’une série

- **Déclaration d’un incendie** : décrit le processus complet d’enregistrement d’un incendie par un forestier sur la carte.

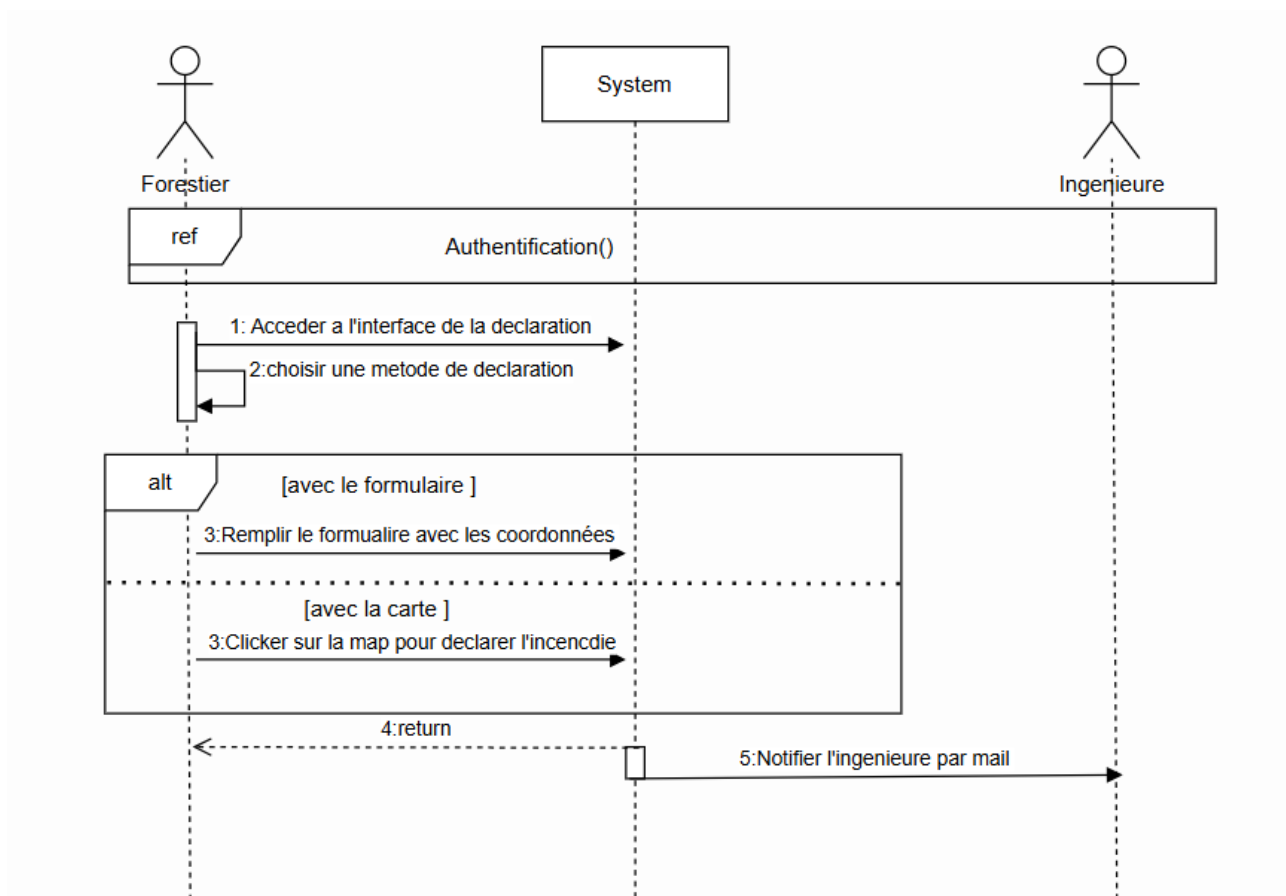


FIGURE 23 : Diagramme de séquence — Déclaration d'un incendie

3.2.3 Architecture générale du système

L'architecture générale du système repose sur le modèle MVC (Modèle – Vue – Contrôleur), renforcé par une séparation nette entre le backend (Laravel) et le frontend (React + TerriaJS). Cette organisation modulaire favorise la maintenabilité, l'évolutivité et la réutilisabilité des composants logiciels.

- **Modèle (Model)** : cette couche gère la logique métier et l'accès aux données. Elle est constituée de modèles Laravel qui interagissent directement avec la base de données PostgreSQL enrichie de l'extension PostGIS pour le traitement des données spatiales. Les entités principales incluent les utilisateurs, les couches spatiales (GeoJSON, raster), les alertes d'incendie et les catalogues.
- **Vue (View)** : la vue est assurée par un frontend développé avec React. Ce dernier offre une interface utilisateur dynamique, fluide et réactive. La visualisation des données spatiales est intégrée à l'aide de la bibliothèque TerriaJS, qui permet d'afficher et d'interagir avec des couches cartographiques riches.
- **Contrôleur (Controller)** : les contrôleurs Laravel servent d'intermédiaires entre la vue et le modèle. Ils reçoivent les requêtes HTTP issues du frontend, traitent la logique métier, effectuent

les requêtes nécessaires via les modèles, et renvoient les résultats sous forme de réponses JSON à l'interface React.

L'interaction entre ces couches se déroule selon le processus suivant :

0. L'utilisateur interagit avec l'interface graphique développée avec React.
0. Les actions de l'utilisateur (consultation de cartes, envoi de formulaires, déclaration d'alertes, etc.) déclenchent des appels API via Axios vers le backend Laravel.
0. Les contrôleurs Laravel reçoivent la requête, invoquent les modèles pour accéder aux données, puis renvoient la réponse structurée.
0. Les données sont ensuite affichées dans l'interface utilisateur et/ou visualisées sur la carte grâce à TerriaJS/ReactJS (Leaflet Map).

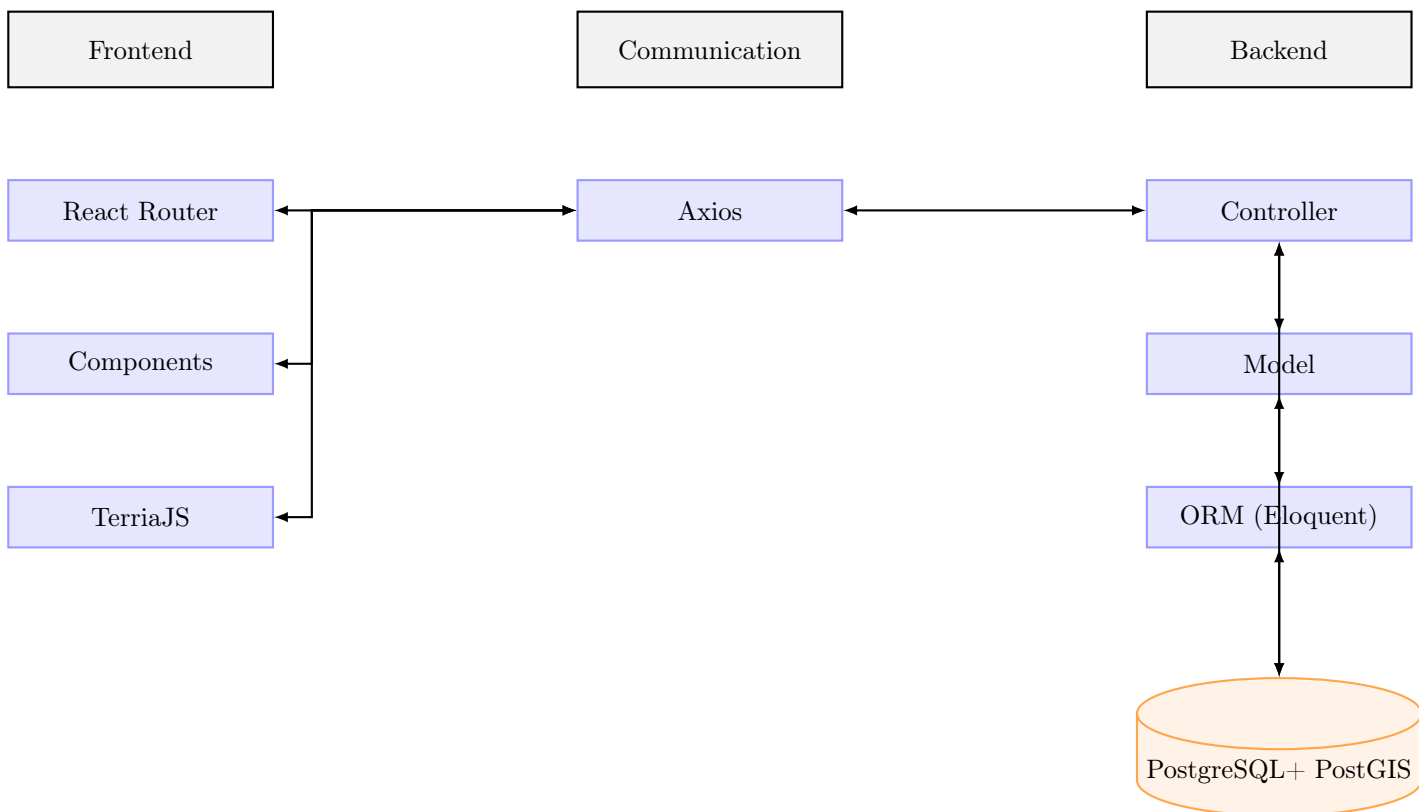


FIGURE 24 : Architecture horizontale du système basé sur le modèle MVC

Cette architecture moderne garantit une séparation des responsabilités, une meilleure sécurité, ainsi qu'une extensibilité du système à long terme.

3.3 Développement backend (Laravel)

Le backend de l'application a été entièrement développé avec le framework **Laravel**, choisi pour sa conformité au modèle MVC, sa richesse fonctionnelle et sa capacité à intégrer efficacement des

bases de données spatiales (*PostgreSQL* avec l'extension *PostGIS*). Il expose une API REST sécurisée, destinée à être consommée par le frontend développé en *React* et la bibliothèque de visualisation *TerriaJS*.

3.3.1 Structure du code

L'organisation du code backend suit l'architecture **MVC** (*Model – View – Controller*) :

- **Routes API** (`routes/api.php`) : elles définissent les points d'entrée de l'application. Chaque route est liée à une méthode d'un contrôleur spécifique.
- **Contrôleurs** (dans `app/Http/Controllers/`) : ils orchestrent la logique métier. Par exemple :
 - `CatalogueController` : gère les dossiers et fichiers géospatiaux ;
 - `AlerteIncendieController` : traite les alertes signalées par les forestiers ;
 - `DistributionController` : gère les associations entre utilisateurs et séries forestières.
- **Modèles** (dans `app/Models/`) : ils représentent les tables de la base de données via l'ORM *Eloquent*, facilitant les requêtes et la gestion des relations, y compris pour les données spatiales.

3.3.2 API RESTful

Toutes les interactions entre le frontend et le backend s'effectuent via une API REST :

- Les endpoints sont accessibles sous le préfixe `/api/`.
- Les contrôleurs retournent des réponses JSON adaptées à la consommation par *React* et *TerriaJS*.
- Les appels se font côté frontend via `Axios`, de manière asynchrone.

Quelques exemples de routes :

```
Route::get('/distributions', [DistributionController::class, 'index']);
Route::post('/distributions', [DistributionController::class, 'store']);
Route::get('/distributions/serie', [DistributionController::class, 'series']);
Route::get('/distributions/{id}', [DistributionController::class, 'show']);
```

FIGURE 25 : Extrait des routes API définies dans `routes/api.php`

3.3.3 Sécurité et authentification

L'authentification est assurée via **Laravel Passport**, qui implémente le protocole *OAuth2* :

- Chaque utilisateur (administrateur, ingénieur, forestier) dispose d'un jeton d'accès personnel.
- Les routes sensibles sont protégées par des **middlewares** d'authentification (`auth:api`) et de vérification de rôle.
- Les mots de passe sont sécurisés via un mécanisme de hachage (`bcrypt`).

Exemple de groupe de routes sécurisé :

```
route::middleware('auth:api','admin')->group(function () {
    //Route::apiResource('users', 'App\Http\Controllers\UserController');
    Route::get('/users', [UserController::class, 'index']); // Liste des utilisateurs
    Route::post('/users', [UserController::class, 'store']); // Créer un utilisateur
    Route::get('/users/{id}', [UserController::class, 'show']); // Afficher un utilisateur
    Route::put('/users/{id}', [UserController::class, 'update']); // Mettre à jour un utilisateur
    Route::delete('/users/{id}', [UserController::class, 'destroy']); // Supprimer un utilisateur
    Route::get('/users/role/{role}', [UserController::class, 'getUsersByRole']);
});
```

FIGURE 26 : Extrait des routes API sécurisées définies dans `routes/api.php`

3.3.4 Interactions avec la base de données spatiale

Le backend exploite une base de données **PostgreSQL** enrichie avec l'extension **PostGIS**, dédiée à la gestion des objets géographiques. L'interaction avec ces données s'effectue à travers :

- Des requêtes SQL brutes intégrant des fonctions spatiales :

```
$features = DB::select('
SELECT
    id,
    "NOM_SERIE",
    "SUBD",
    "NOM_TRIAGE",
    "SURFACE",
    "PERIMETRE",
    ST_AsGeoJSON(ST_Transform(geom,4326))::json AS geometry
FROM
    series_a_accorde;
');
```

FIGURE 27 : Exemple de requête spatiale avec PostGIS dans l'IDE

- Des modèles *Eloquent* adaptés à la structure des tables spatiales.
- Des méthodes de transformation générant des objets **GeoJSON** directement exploitables côté frontend.

3.3.5 Fonctionnalités serveur implémentées

Le backend intègre plusieurs fonctionnalités spécifiques orientées géospatiales :

- **CRUD Utilisateurs** : gestion complète des utilisateurs (création, lecture, mise à jour, suppression) via des routes API sécurisées. Chaque utilisateur est associé à un rôle (administrateur, ingénieur, forestier), avec des droits d'accès contrôlés par des middlewares.
- **Catalogue dynamique** : gestion hiérarchique de fichiers et dossiers, avec import automatique de fichiers `.geojson`, extraction et stockage dans des tables PostgreSQL.
- **Système d'alerte incendie** : les forestiers peuvent signaler un feu via une carte interactive. L'alerte est enregistrée et notifiée aux ingénieurs .

- **API GeoJSON** : les couches spatiales (routes, douars, séries forestières, etc.) sont exposées sous forme de fichiers **GeoJSON** générés dynamiquement.
- **Distribution des séries forestières** : chaque utilisateur forestier est associé à une ou plusieurs séries, avec un affichage restreint selon ses droits d'accès.

Cette architecture backend modulaire, sécurisée et centrée sur les données géospatiales garantit une communication efficace avec le frontend, tout en assurant extensibilité et maintien de l'intégrité des données.

3.4 Développement frontend (React & TerriaJS)

L'interface utilisateur de l'application a été développée en s'appuyant sur les technologies **React** pour la gestion des interfaces dynamiques et **TerriaJS** pour la visualisation des données spatiales. Cette combinaison technologique permet de proposer une application web moderne, interactive et évolutive, adaptée à plusieurs profils d'utilisateurs.

3.4.1 Architecture de l'interface utilisateur

L'interface a été conçue selon une structure modulaire, reposant sur des composants *React* réutilisables. L'ensemble du frontend est organisé en plusieurs pages (*dashboards*) dédiées aux différents rôles utilisateurs, chacun ayant des permissions et des vues spécifiques :

- **Administrateur** : gestion des utilisateurs, des catalogues, des fichiers et supervision globale du système.
- **Ingénieur** : réception des alertes d'incendie, suivi des zones déclarées et accès aux données géospatiales pertinentes.
- **Forestier** : visualisation de la série forestière qui lui est attribuée, consultation des données spatiales, déclaration d'incendie sur la carte.

Chaque page est encapsulée dans un layout spécifique, assurant cohérence visuelle, facilité de navigation et séparation des responsabilités.

3.4.2 Intégration de TerriaJS pour la visualisation cartographique

La bibliothèque **TerriaJS** a été intégrée au sein de composants React personnalisés afin d'assurer le rendu cartographique des données spatiales. Cette intégration permet de charger dynamiquement des couches spatiales issues du backend sous forme de fichiers **GeoJSON**.

Les fonctionnalités cartographiques mises en œuvre incluent :

- Affichage des séries forestières, routes, douars et vigies sous forme de couches superposées.
- Visualisation des fichiers spatiaux stockés dans le système.
- Interaction avec la carte pour la déclaration d’incendie (clic sur une zone géographique).
- Sélection dynamique des couches à afficher via un catalogue configurable.

TerriaJS fournit également une interface utilisateur intégrée (gestion des couches, légendes, zoom, sélection), permettant de réduire le temps de développement et d’améliorer l’expérience utilisateur.

3.4.3 Justification du choix de TerriaJS

Le choix de **TerriaJS** pour la visualisation cartographique repose sur une série d’avantages significatifs par rapport à d’autres bibliothèques telles que *Leaflet*, *OpenLayers* ou *Mapbox GL* :

- Compatibilité étendue avec des formats de données géospatiales standards (*GeoJSON*, *KML*, *WMS*, *WFS*), facilitant l’intégration directe de fichiers issus de différentes sources.
- Chargement dynamique de couches via un système de catalogue, rendant possible l’activation ou la désactivation de données sans redéploiement.
- Interface utilisateur complète intégrée nativement : gestion des couches, filtres, métadonnées, zooms, outils de recherche.
- Interopérabilité avec les APIs REST : les couches peuvent être générées dynamiquement côté serveur et intégrées dans la carte à la volée.
- Réduction du temps de développement : l’environnement de TerriaJS propose une base solide et modulaire, minimisant les efforts liés à la création d’une interface cartographique professionnelle.

Ces caractéristiques font de **TerriaJS** une solution parfaitement adaptée aux exigences du projet, notamment en matière de visualisation de données spatiales interactives et de flexibilité d’intégration avec une API Laravel.

3.4.4 Communication avec le backend

L’ensemble des échanges entre le frontend et le backend s’effectue via des appels API REST sécurisés, en utilisant la bibliothèque **Axios**. Chaque action utilisateur déclenche un appel API (création, récupération, mise à jour ou suppression de données), les réponses étant ensuite exploitées pour alimenter l’état de l’application via les hooks React (`useState`, `useEffect`).

Exemples d’interactions :

- Récupération des couches *GeoJSON* à afficher sur la carte.
- Envoi des données d’alerte incendie depuis le forestier vers le backend.

- Récupération et affichage dynamique des utilisateurs et catalogues dans les dashboards.

Un système d'authentification basé sur **Laravel Passport** est utilisé pour sécuriser les requêtes, avec gestion automatique des tokens d'accès.

3.4.5 Fonctionnalités principales développées côté interface

Le frontend intègre plusieurs fonctionnalités majeures assurant l'interactivité et la gestion complète de l'application :

- **Authentification et redirection par rôle** : chaque utilisateur est dirigé vers un tableau de bord adapté à son profil après connexion.
- **CRUD Utilisateurs** : l'administrateur peut gérer l'ensemble des comptes via une interface intuitive (formulaires et tableaux dynamiques).
- **Affichage des couches spatiales** : chargement et affichage dynamique des données spatiales sous forme de couches superposées (GeoJSON, GeoTIFF).
- **Déclaration d'incendie** : le forestier peut cliquer sur la carte pour déclarer un incident géographique, transmis à l'ingénieur en temps réel.
- **Interface fluide et réactive** : mise à jour instantanée de l'interface sans rechargement de page, grâce au système de composants et au routage client de React.

Cette approche frontend moderne assure une interaction conviviale avec le système, tout en exploitant pleinement la puissance de visualisation offerte par **TerriaJS** et la modularité de **React**.

3.5 Extraction des valeurs raster pour la data science

Dans le cadre de la modélisation des incendies de forêt, l'enrichissement des bases de données avec des variables géospatiales dérivées de couches raster est une étape essentielle. Ces variables, extraites automatiquement à partir de couches SIG (altitude, pente, proximité de routes, proximité de routes douars etc.), permettent de fournir aux modèles de data science des indicateurs précis à haute résolution spatiale.

3.5.1 Objectif de l'extraction raster

L'objectif principal est d'associer à chaque incendie (contenu dans les tables **dgf**, **nasa**, **effis**) un ensemble de valeurs issues de couches raster pertinentes. Ces valeurs enrichissent les données tabulaires avec des attributs environnementaux et d'infrastructure, utiles pour l'analyse statistique, la prédiction et la modélisation.

3.5.2 Méthodologie générale

Pour chaque base d’incendies (DGF, NASA, EFFIS), deux scripts Python ont été développés :

- **Script individuel** : destiné à enrichir un incendie spécifique (test et exemple visuel).
- **Script de masse** : pour mettre à jour tous les enregistrements de la table.

Les étapes principales sont les suivantes :

- Connexion à la base de données PostgreSQL/PostGIS.
- Vérification et création des colonnes nécessaires (si absentes).
- Définition de dictionnaires de mappage pour les codes de forêt.
- Boucle sur chaque incendie pour :
 - Extraire la valeur raster au point (centroïde de la géométrie),
 - Convertir certains codes en descriptions,
 - Mettre à jour dynamiquement la base.

3.5.3 Couches raster utilisées

Chaque script s’appuie sur une correspondance entre nom de colonne cible et table raster source.

Voici quelques exemples de couches utilisées :

- **Caractéristiques physiques** : `altitude_raster`, `pente_raster`, `ombrage_raster`, `exposition_raster`
- **Proximité d’infrastructures** : `proximite_reseau_routier_raster`, `proximite_zones_urbaines_raster`, etc.
- **Environnement forestier** : `composition_de_for_ts_raster`, `type_foret_raster`, etc.

3.5.4 Exemple de fonction d’extraction

Chaque script utilise une fonction SQL intégrée à PostGIS. Par exemple, la fonction `ST_Value()` permet d’extraire une valeur raster à partir de coordonnées géographiques, comme illustré dans la Figure 28.

```
def get_raster_value(conn, table_name, x, y, band=1):
    cur.execute(f"""
        SELECT ST_Value(
            rast, %s,
            ST_SetSRID(ST_MakePoint(%s, %s), 32632)
        )
        FROM "{table_name}"
        WHERE ST_Intersects(
            rast,
            ST_SetSRID(ST_MakePoint(%s, %s), 32632)
        )
        LIMIT 1;
    """, (band, x, y, x, y))

    result = cur.fetchone()
    return float(result[0]) if result and result[0] is not None else None
```

FIGURE 28 : Fonction Python `get_raster_value` pour extraire une valeur raster à partir des coordonnées.

Le script Python formate dynamiquement cette requête pour chaque couche raster, puis met à jour la table correspondante via une fonction de mise à jour, comme montré dans la Figure 29.

```
def update_incendie(conn, incendie_id):
    if updates:
        with conn.cursor() as cur:
            try:
                set_clause = ", ".join(f"{k}" = %s' for k in updates)
                values = list(updates.values()) + [incendie_id]
                cur.execute(f"""
                    UPDATE dgf
                    SET {set_clause}
                    WHERE id = %s
                """, values)
                conn.commit()
                print(f" Mise à jour {len(updates)} valeurs raster")
            except Exception as e:
                conn.rollback()
                print(f" Erreur mise à jour raster: {e}")
```

FIGURE 29 : Fonction `update_incendie` pour mettre à jour dynamiquement les attributs raster dans la base de données.

- Structure plus claire avec des paragraphes séparés
- Explication plus précise de la fonction `ST_Value()`
- Correction de la ponctuation et de la mise en forme
- Harmonisation des termes techniques
- Liaison logique entre les deux figures

La Figure 30 présente un extrait du script Python permettant d'enrichir dynamiquement l'ensemble des incendies enregistrés dans la table `dgf`. Cette opération est réalisée en appelant de manière itérative le script `enrich_dgf_one.py` pour chaque identifiant d'incendie (`id`) récupéré depuis la base de données.

```

enrich_dgf.py > ...

cur.execute("SELECT id FROM dgf")
ids = cur.fetchall()

python_path = os.path.join(os.getcwd(), '.venv', 'Scripts', 'python.exe')

for (inc_id,) in ids:
    print(f" Enrichissement de l'incendie {inc_id}")
    result = subprocess.run(
        [python_path, 'scripts\\enrich_dgf_one.py', str(inc_id)],
        capture_output=True, text=True
    )

    print(result.stdout)
    if result.stderr:
        print(" Erreur :", result.stderr)

cur.close()
conn.close()

```

FIGURE 30 : Fonction `enrich_dgf.py` pour enrichir tous les incendies de la table `dgf`.

3.5.5 Avantages de l'approche

- **Automatisation complète** : les scripts gèrent la création des colonnes manquantes.
- **Modularité** : les scripts sont facilement adaptables à d'autres jeux de données.
- **Préparation à la modélisation** : les données raster enrichies constituent les variables indépendantes dans les modèles de machine learning.

3.5.6 Résumé des scripts

Script	Table cible	Mode	Description
<code>enrich_dgf_one.py</code>	<code>dgf</code>	Individuel	Enrichissement d'un seul incendie
<code>enrich_dgf.py</code>	<code>dgf</code>	En masse	Enrichissement total table DGF
<code>enrich_effis_one.py</code>	<code>effis</code>	Individuel	Enrichissement d'un seul incendie
<code>enrich_effis.py</code>	<code>effis</code>	En masse	Enrichissement total table EFFIS
<code>enrich_nasa_one.py</code>	<code>nasa</code>	Individuel	Enrichissement d'un seul incendie
<code>enrich_nasa.py</code>	<code>nasa</code>	En masse	Enrichissement total table NASA

TABLEAU 9 : Liste des scripts d'extraction raster développés

3.6 Modélisation Data Science

L'objectif de la modélisation data science est de prédire le risque d'incendie et la surface potentiellement brûlée en se basant sur des facteurs environnementaux, topographiques, climatiques et d'accessibilité.

3.6.1 Construction de la base d'apprentissage

La base d'apprentissage a été constituée à partir de données spatiales extraites pour chaque pixel de la zone d'étude. Les variables explicatives utilisées comprennent :

- **Données topographiques** : pente, altitude, orientation (exposition).
- **Données environnementales** : type et composition de la forêt , type de sol.
- **Données temporelles** : date de l'incendie .
- **Données d'accessibilité et de proximité** : distance aux routes, frontières, infrastructures hydrauliques (barrages, forages, lacs), zones urbaines et vigies...
- **Variables climatiques agrégées** :
 - Température journalière
 - Humidité journalière
 - Vitesse du vent annuelle
 - Direction du vent annuelle
 - Précipitations annuelles
 - Humidité annuelle
 - Température annuelle
 - Température minimale annuelle
 - Température maximale annuelle

La variable cible pour la régression est la surface brûlée, transformée logarithmiquement pour stabiliser la variance et normaliser sa distribution. Pour la classification, une version binarisée a été utilisée .

Les données raster ont été croisées avec les polygones d'incendies historiques pour constituer le jeu de données. Un prétraitement a été appliqué :

- **Gestion des données manquantes** : imputation par la médiane pour les variables numériques et par le mode (valeur la plus fréquente) pour les variables catégorielles. Les lignes où la variable cible 'SURFACE' était nulle ou manquante ont été supprimées pour la modélisation de la surface brûlée.

- **Traitement des outliers** : pour les variables continues, les outliers ont été capés en utilisant la méthode de l'intervalle interquartile (IQR).
- **Normalisation** : les variables numériques ont été standardisées (Z-score) pour homogénéiser les échelles.
- **Encodage** : les variables catégorielles ont été encodées en utilisant One-Hot Encoding.

3.6.2 Méthodes utilisées

Trois approches principales ont été explorées : le clustering pour identifier des profils météorologiques, la classification du risque d'incendie, et la régression de la surface brûlée.

- **Clustering (K-Means)** : Préalablement à la modélisation principale, un algorithme **K-Means** a été appliqué sur les variables dynamiques (climatiques et temporelles) afin d'identifier des profils météorologiques et/ou écologiques distincts au sein de la zone d'étude. L'analyse a montré que 3 clusters suffisaient à segmenter les données de manière pertinente. Cette information de cluster a ensuite été intégrée comme une variable explicative catégorielle dans les modèles de classification et de régression.
- **Classification du risque d'incendie** : L'objectif est de prédire si une zone présente un risque d'incendie significatif (classification binaire basée sur 'SURFACE classe binaire'). Les algorithmes testés incluent **Random Forest Classifier**, **Gradient Boosting Classifier (XGBoost)** et **Support Vector Machine (SVM)**. Le modèle final retenu est un **Random Forest Classifier**, en raison de sa robustesse, sa capacité à modéliser les interactions complexes et son interprétabilité (importance des variables).
- **Régression de la surface brûlée** : L'objectif est de prédire la valeur continue de la surface brûlée (transformée logarithmiquement, 'log(SURFACE)'). Plusieurs modèles ont été évalués, notamment la **régression linéaire multiple**, le **Random Forest Regressor** et le **Gradient Boosting Regressor (XGBoost)**. Le **XGBoost Regressor** s'est révélé le plus performant. Une sélection de variables basée sur l'importance des features a été effectuée pour optimiser ce modèle.

L'implémentation a été réalisée à l'aide des bibliothèques Python `scikit`, `xgboost` et `pandas` pour la manipulation des données tabulaires, en complément de `rasterio` et `geopandas` pour les aspects spatiaux et `matplotlib` pour la visualisation.

3.6.3 Évaluation des modèles

Les modèles ont été évalués à l'aide de plusieurs métriques, après une division des données en ensembles d'entraînement et de test. Une recherche par grille (GridSearchCV) avec validation croisée en k -fold (avec $cv = 5$) a été utilisée pour optimiser les hyperparamètres et assurer la robustesse des performances.

— **Pour la classification (Random Forest) :**

- Accuracy
- F1-score
- Courbe ROC et aire sous la courbe (AUC)

Les résultats typiques obtenus après optimisation étaient une accuracy d'environ 85% et une AUC de 0.91.

— **Pour la régression (XGBoost Regressor) :**

- Erreur quadratique moyenne (RMSE) sur la cible 'log(SURFACE)'.
- Coefficient de détermination (R^2).
- Erreur absolue moyenne (MAE).

Le modèle XGBoost optimisé a atteint un R^2 d'environ 0.39 et un RMSE de 2.63 sur l'échelle log-transformée. Bien que le R^2 puisse paraître modeste, la prédiction de phénomènes naturels complexes comme les incendies est intrinsèquement difficile. L'amélioration de la qualité et de la granularité des données d'entrée est une piste pour de meilleures performances.

3.7 Résultats de la modélisation

Après l'entraînement des modèles sur les données extraites, des cartes de prédiction ont été générées à l'échelle pixel pour chaque cluster météorologique identifié. Ces visualisations permettent d'anticiper les risques et impacts potentiels selon la localisation géographique et le profil météorologique.

3.7.1 Cartes des zones à risque (Classification par cluster)

Le modèle Random Forest Classifier a permis d'assigner à chaque pixel une classe de risque (par exemple, Inférieure ou égale à 1 ha/Supérieure à 1 ha). Les cartes suivantes illustrent ces prédictions pour les trois clusters météorologiques distincts au sein de la zone d'étude de Jendouba.

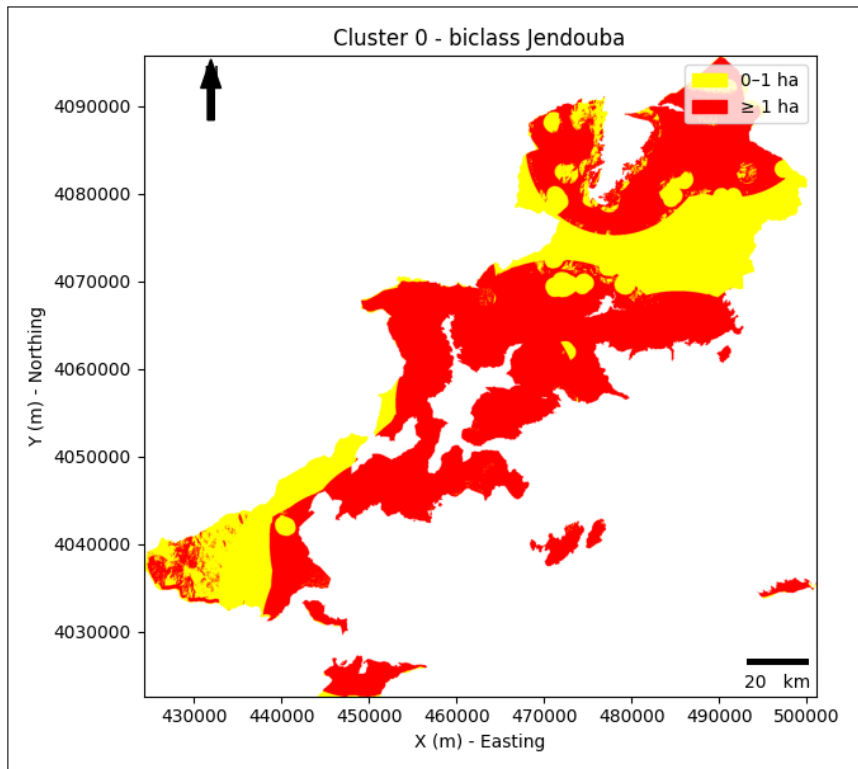


FIGURE 31 : Carte des zones à risque d'incendie – Cluster 0 (Jendouba)

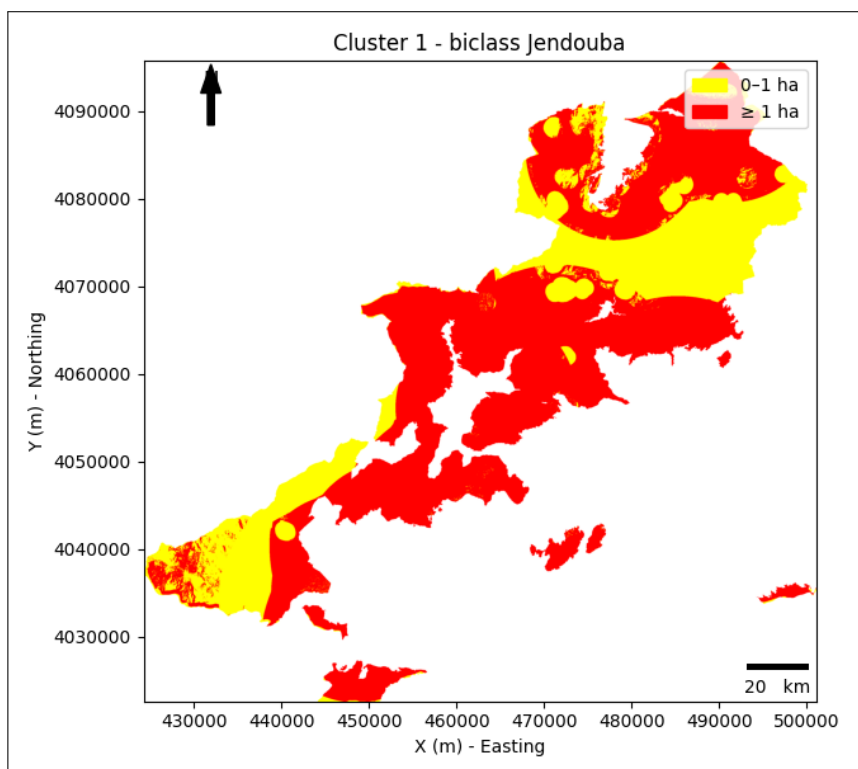


FIGURE 32 : Carte des zones à risque d'incendie – Cluster 1 (Jendouba)

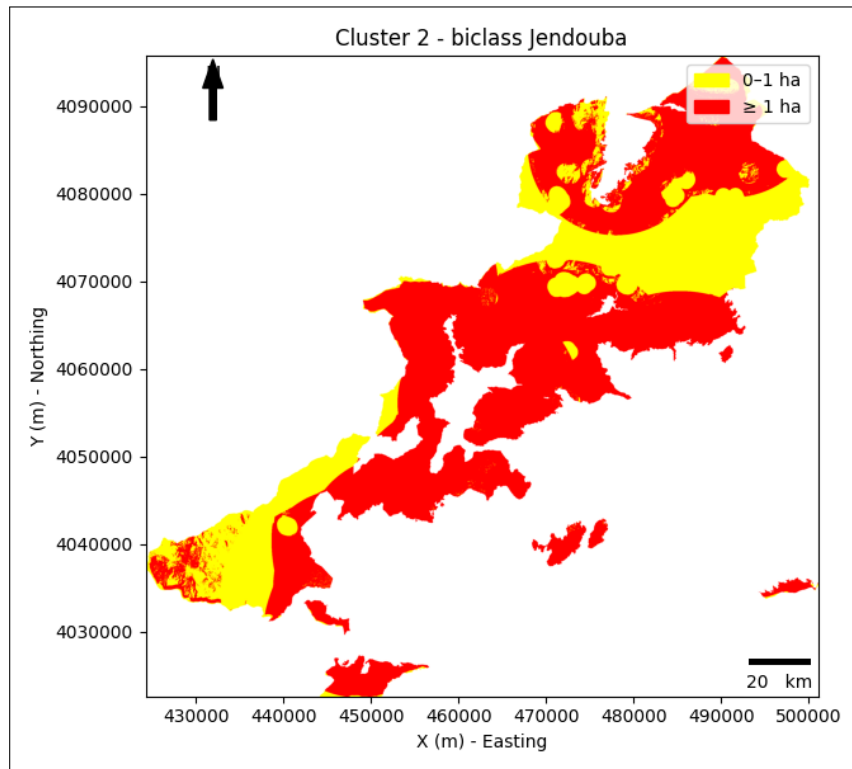


FIGURE 33 : Carte des zones à risque d'incendie – Cluster 2 (Jendouba)

Analyse :L’analyse croisée des trois clusters météorologiques révèle une stabilité spatiale des zones à risque élevé, localisées principalement dans les massifs forestiers à forte pente et dense couverture végétale. Ces zones demeurent critiques quel que soit le scénario climatique, bien que le Cluster 2 (conditions plus sèches et venteuses) accentue leur étendue. À l’inverse, les zones agricoles, urbaines ou faiblement végétalisées sont systématiquement associées à un risque réduit. Cette classification met en évidence l’interaction entre facteurs structurels (topographie, usage du sol) et conditions météorologiques, soulignant l’importance d’une gestion différenciée du territoire selon les scénarios climatiques.

3.7.2 Cartes de prédiction de surface brûlée (Régression par cluster)

Le modèle XGBoost Regressor permet de cartographier la surface potentielle brûlée à l’échelle pixel. Les cartes suivantes montrent ces estimations pour les trois clusters météorologiques.

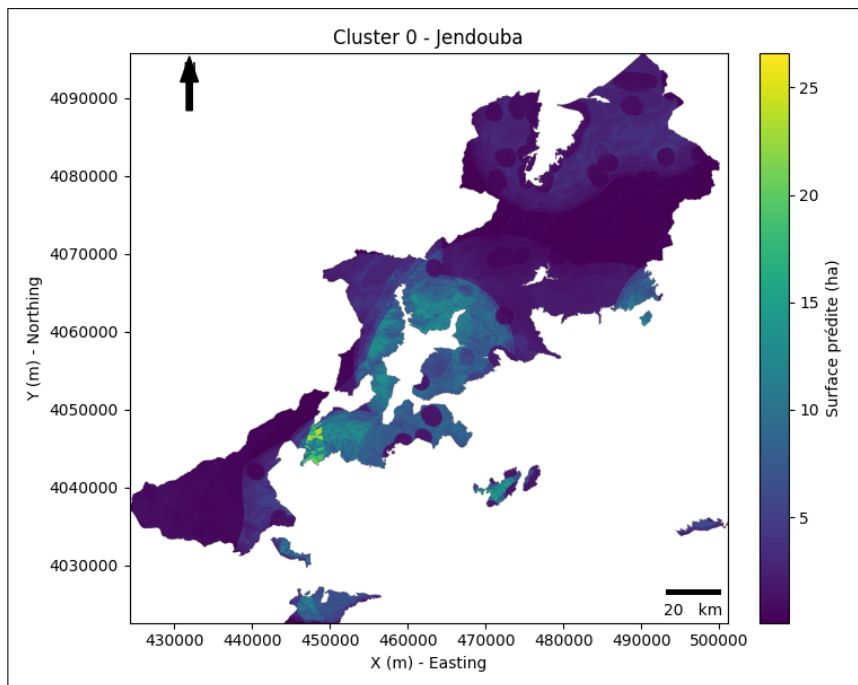


FIGURE 34 : Carte estimée de la surface brûlée (ha) – Cluster 0 (Jendouba)

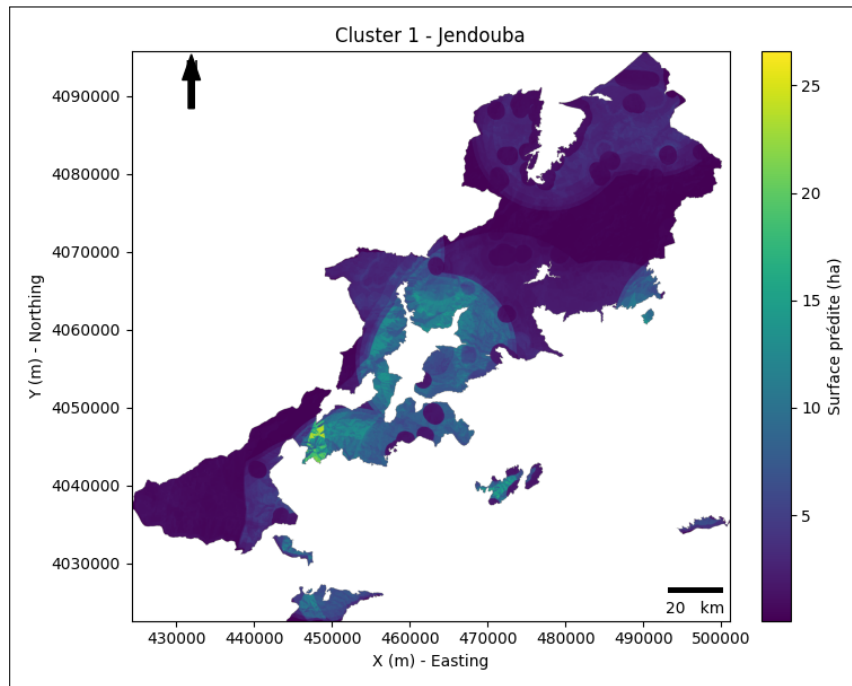


FIGURE 35 : Carte estimée de la surface brûlée (ha) – Cluster 1 (Jendouba)

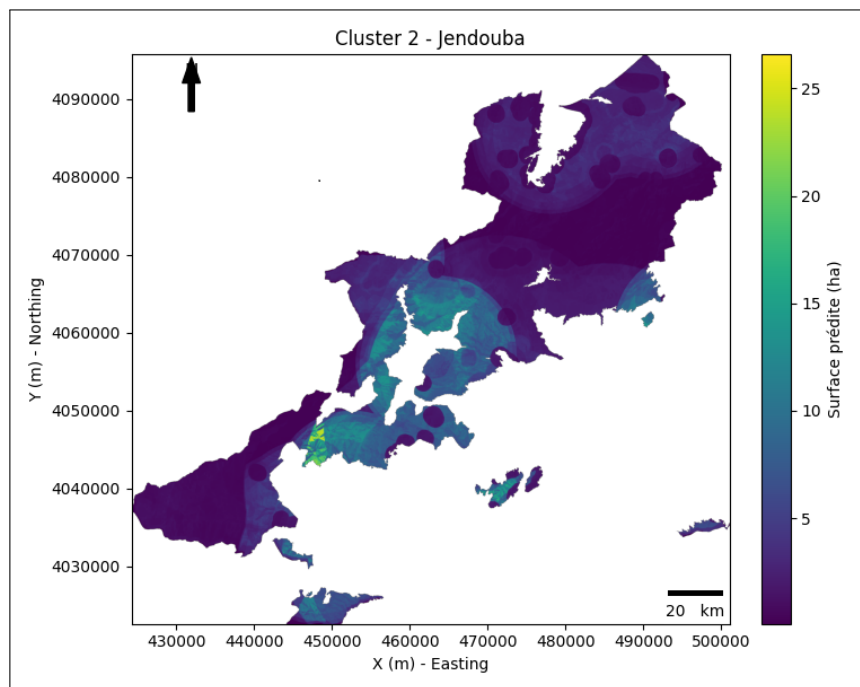


FIGURE 36 : Carte estimée de la surface brûlée (ha) – Cluster 2 (Jendouba)

Analyse : Les prédictions de surface brûlée indiquent une concentration des valeurs élevées dans les zones densément végétalisées, éloignées des infrastructures d'accès rapide. La variabilité spatiale entre clusters montre une sensibilité accrue sous les conditions du Cluster 2, avec une extension des zones potentiellement affectées. Ces résultats soulignent l'importance des interactions entre conditions météorologiques extrêmes, accessibilité et charge combustible dans la dynamique des incendies.

3.8 Intégration des résultats dans le web mapping

3.8.1 3.8 Intégration des résultats dans le web mapping

L'objectif de cette section est de présenter la manière dont les résultats de la modélisation prédictive ont été intégrés dans le système cartographique interactif de l'application *Wildfire Map*. L'originalité de cette intégration repose sur l'utilisation combinée d'un modèle de clustering, de cartes raster thématiques (issues de modèles de régression ou de classification) et d'un mécanisme interactif permettant à l'ingénieur forestier d'obtenir des informations précises sur les zones à risque directement depuis la carte.

3.8.1.1 3.8.1 Intégration statique des cartes de résultats

Trois clusters d'incendies ont été identifiés à l'aide d'une analyse de clustering (par exemple, KMeans) basée sur les données météorologiques et historiques. Pour chaque cluster, deux modèles prédictifs ont été entraînés :

- Un modèle de **classification** qui prédit la classe de risque (par exemple : faible, moyen, élevé).
- Un modèle de **régression** qui estime la surface brûlée.

Les résultats de chaque modèle ont été exportés sous forme de cartes raster géoréférencées (GeoTIFF). Ainsi, chaque cluster possède deux cartes raster, soit un total de six cartes raster prédictives.

Ces fichiers raster sont ensuite stockés dans une base de données **PostgreSQL/PostGIS**, puis servis via **Laravel** en tant que couches raster dans l'interface **Leaflet**.

3.8.1.2 3.8.2 Intégration dynamique via API Flask

L'intégration dynamique permet d'exploiter un modèle d'apprentissage automatique en temps réel pour analyser des données météorologiques saisies dans l'application, et afficher une carte prédictive appropriée sur la carte web. Cette approche repose sur une collaboration entre **Laravel** et un microservice **Python (API Flask)** dédié aux traitements de type *Data Science*.

Fonctionnement détaillé :

1. **Saisie des données météorologiques :** L'ingénieur forestier saisit les données météo (température, humidité, vitesse du vent, etc.) via un formulaire React. Il sélectionne également le type de modèle souhaité :
 - **Classification** → prédit la classe de l'incendie (faible, moyen, fort).
 - **Régression** → estime la surface brûlée attendue.

2. **Transmission à Laravel** : Ces données sont envoyées au backend Laravel via une requête Axios. Laravel valide et formate les données.
3. **Appel à l'API Flask (modèle de clustering)** : Laravel envoie ensuite une requête HTTP POST à une API Flask hébergeant le modèle de clustering. Cette API retourne le cluster auquel appartiennent les conditions météo saisies.
4. **Traitement du résultat dans Laravel** : Laravel utilise la réponse de Flask (ex. : cluster 2) et le choix utilisateur (régression ou classification) pour :
 - Identifier la carte raster prédictive à afficher (ex. : `cluster2_regression.tif`).
 - Fournir l'URL de cette couche raster au frontend React.
5. **Affichage dynamique de la carte raster** : React/Leaflet affiche dynamiquement la couche raster prédite en superposition sur la carte.
6. **Lecture de la valeur du pixel au niveau de l'incendie** : Lorsque l'utilisateur clique pour signaler un incendie, Laravel exécute un script Python (via `shell_exec`, `process` ou API) qui lit la valeur du pixel à la position cliquée sur la carte raster active :
 - Si classification → on lit la classe de l'incendie (plus ou moins d'un ha).
 - Si régression → on lit la valeur estimée de la surface brûlée (ex. : 72.5 ha).
7. **Affichage de l'information dans la popup de l'incendie** : La valeur lue dans la carte raster est renvoyée à React pour être affichée dans la popup de l'incendie.

Architecture technique

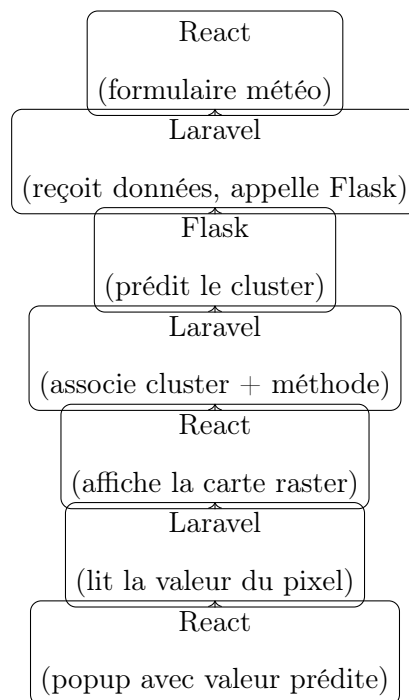


FIGURE 37 : Architecture technique corrigée entre React, Laravel et Flask

Tableau des figures associées

Figure	Description
Figure 3.8.1	Formulaire React de saisie météo avec option classification/régression
Figure 3.8.2	Affichage dynamique d'une couche raster selon cluster + méthode
Figure 3.8.3	Popup d'un incendie montrant la classe ou la surface prédite
Figure 3.8.4	Illustration du flux d'architecture entre React, Laravel et Flask
Figure 3.8.5 *(optionnel)*	Code Laravel de consommation de l'API Flask

TABLEAU 10 : Résumé des figures associées à l'intégration web mapping

3.9 Visualisation des interfaces utilisateur

L'interface utilisateur de l'application *Wildfire Map* a été conçue pour être simple, intuitive et adaptée aux rôles des utilisateurs. Elle permet une interaction fluide avec les données spatiales, les couches raster prédictives et les fonctionnalités d'analyse.

Chaque interface a été pensée selon les besoins spécifiques des utilisateurs cibles (forestiers et ingénieurs), tout en assurant une cohérence visuelle et fonctionnelle à l'échelle du système.

3.9.0.1 Interface de connexion

L'interface de connexion permet d'accéder à l'application selon le profil utilisateur. Elle comprend :

- Deux champs : adresse e-mail et mot de passe ;
- Un système de vérification et de redirection en fonction du rôle (forestier ou ingénieur) ;
- Une interface épurée avec retour d'erreur en cas d'identifiants incorrects.

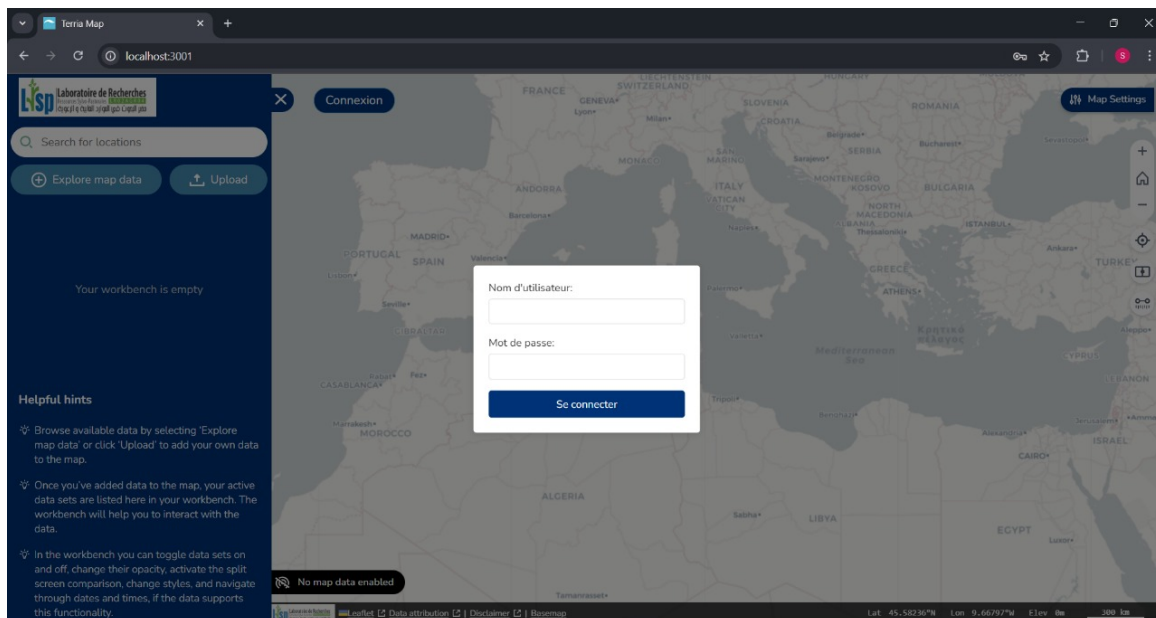


FIGURE 38 : Interface de connexion de l'utilisateur

3.9.0.2 Tableaux de bord des utilisateurs (admin, forestier, ingénieur, visiteur)

L'application *Wildfire Map* propose une gestion multi-utilisateur avec des interfaces adaptées à quatre profils distincts :

Visiteur (tableau de bord simplifié) Le visiteur dispose d'un tableau de bord contenant uniquement des fonctionnalités communes :

- Carte interactive avec couches raster et alertes validées ;
- Statistiques générales sur les incendies ;
- Navigation limitée sans accès aux fonctions avancées (déclaration, gestion, prédiction).

Ce tableau est destiné à une consultation publique des données.

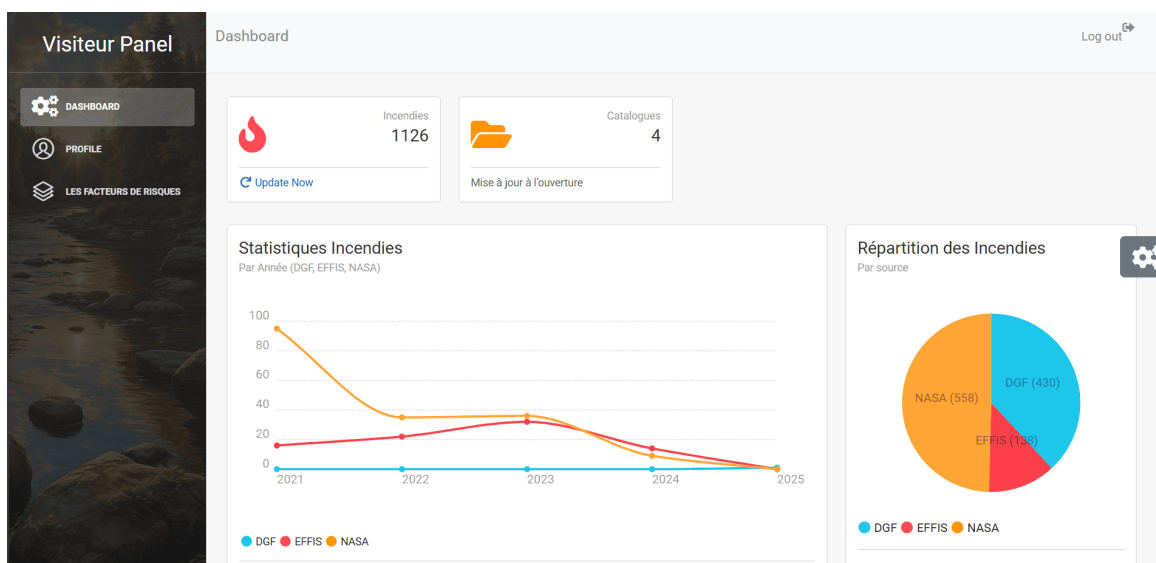


FIGURE 39 : Tableau de bord du visiteur

Forestier Le forestier est en lien direct avec le terrain. Son tableau de bord lui permet de :

- Visualiser sa série forestière affectée sur la carte ;
- Déclarer un incendie en cliquant sur la carte ;
- Suivre l'état de ses déclarations.

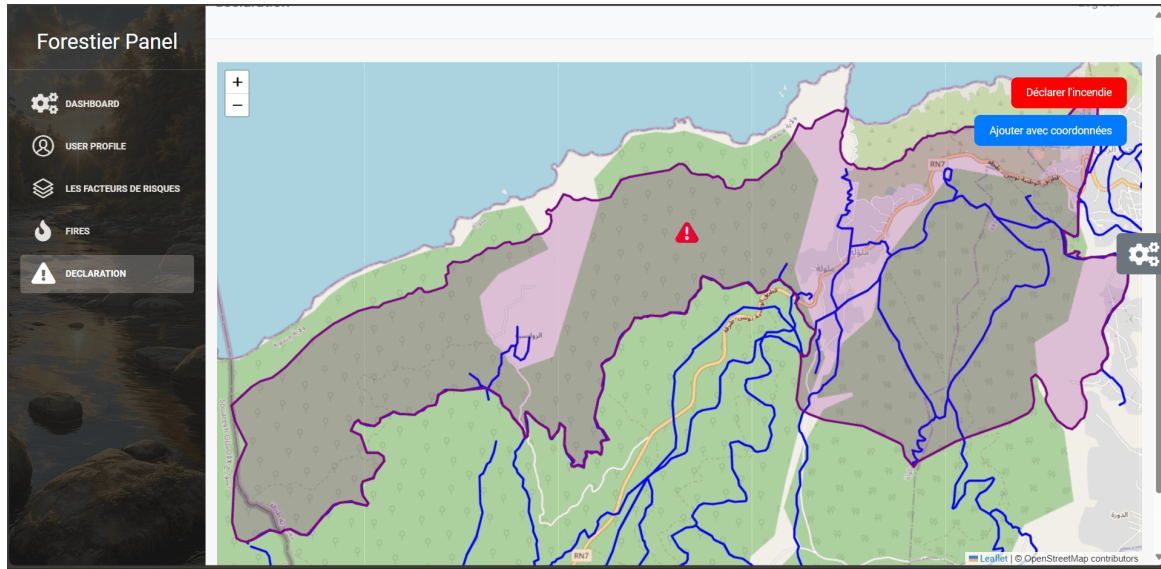


FIGURE 40 : Interface forestier avec déclaration d'incendie et carte des séries

Ingénieur L'ingénieur a accès à des outils d'analyse avancée, notamment :

- Le formulaire météo pour lancer des prédictions ;
- L'affichage dynamique des couches raster (classification ou régression) selon le cluster prédit ;
- Le suivi des alertes incendie reçues depuis le terrain.

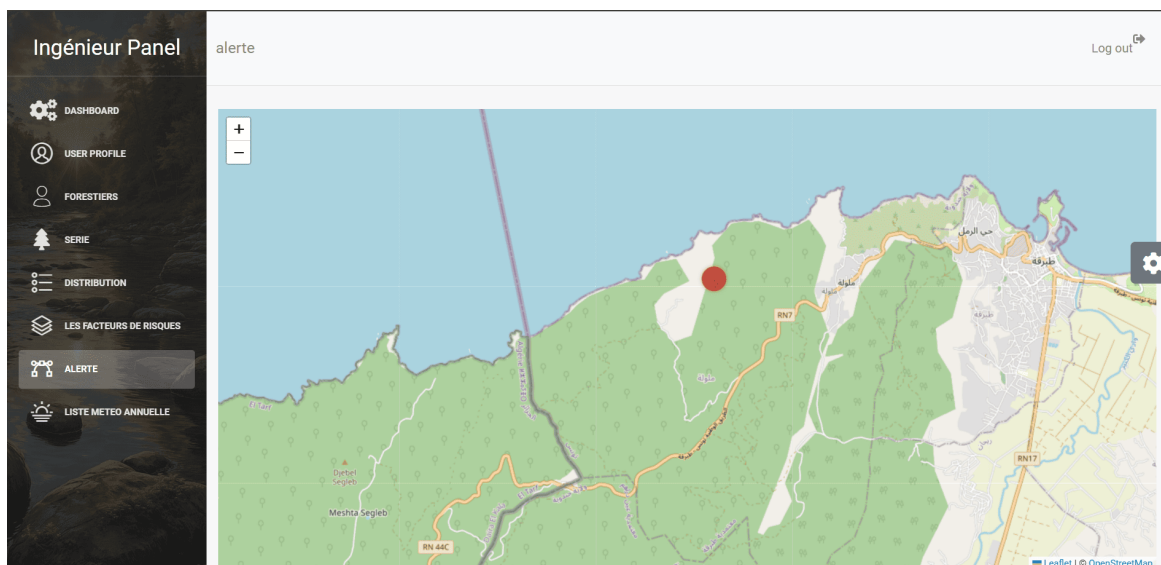


FIGURE 41 : Tableau de bord de l'ingénieur avec prédiction et visualisation raster

Administrateur L'administrateur dispose d'un tableau de bord centralisé lui permettant de :

- Gérer les utilisateurs (création, modification, suppression) ;
- Visualiser toutes les alertes et séries enregistrées ;
- Accéder à des statistiques globales du système.

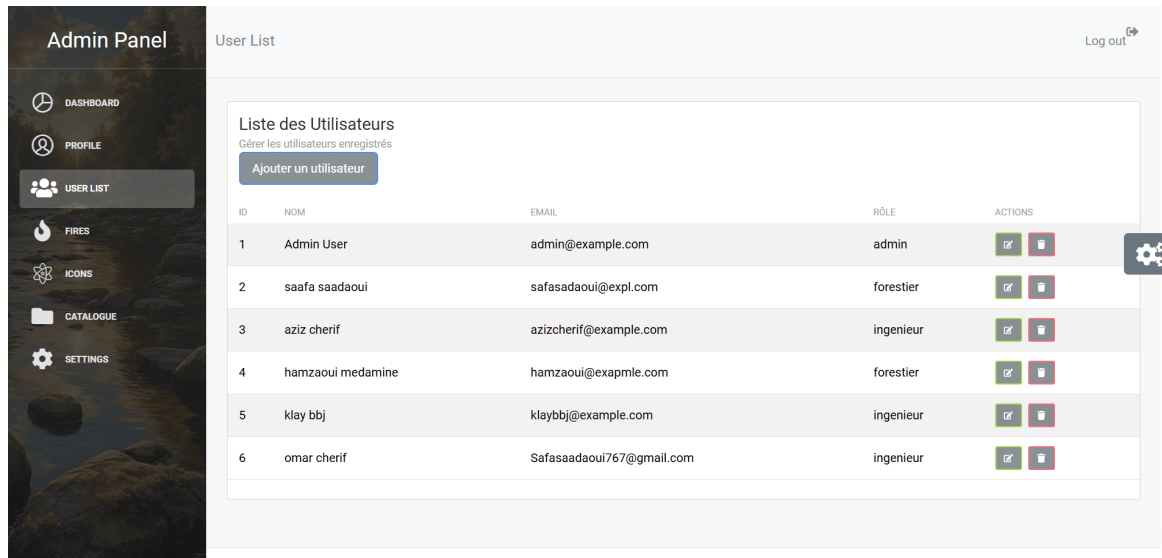


FIGURE 42 : Interface de gestion des utilisateurs par l'administrateur

3.9.0.3 Carte des incendies avec couches superposées

La carte est le cœur de l'interface. Elle est développée avec *Terria.js* et affiche :

- Les incendies déclarés, positionnés manuellement ou automatiquement ;
- Les couches géographiques supplémentaires : routes, douars, vigies, séries.

L'utilisateur peut activer/désactiver les couches, zoomer, et obtenir des informations contextuelles.

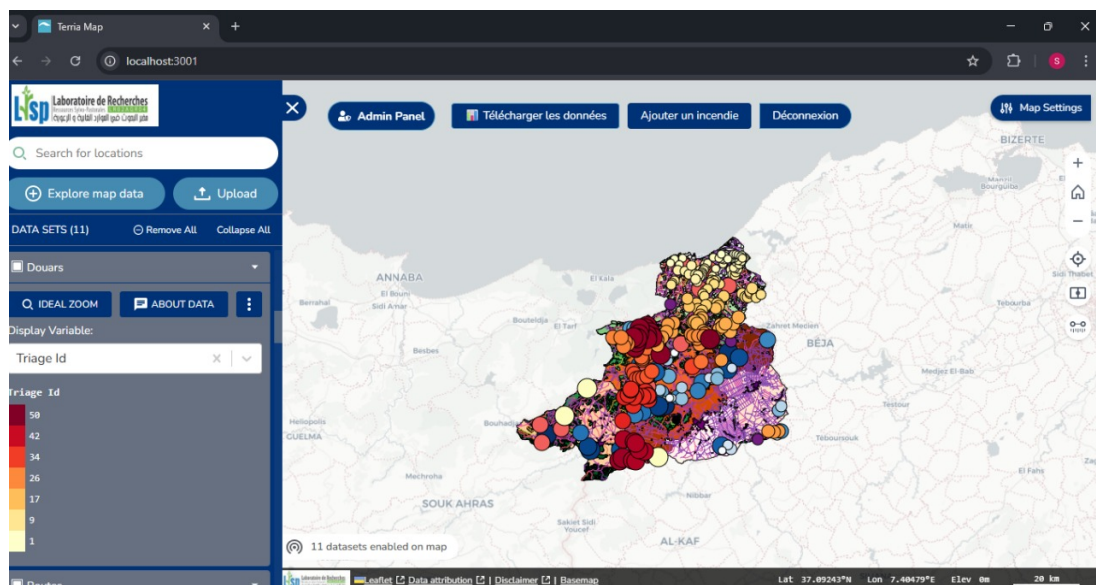


FIGURE 43 : Carte interactive affichant un incendie et une couche raster

3.9.0.4 Formulaire météo et affichage des résultats

Dans le tableau de bord ingénieur, un formulaire permet de :

- Saisir les paramètres météo actuels (température, humidité, vent...);
- Sélectionner le type de prédiction souhaité : classification ou régression;
- Lancer l'analyse pour afficher automatiquement la couche raster correspondante.

Une fois la prédiction effectuée, la carte se met à jour dynamiquement avec la couche adaptée (selon le cluster identifié).

The screenshot displays the 'Ingénieur Panel' interface. On the left is a dark sidebar with navigation links: DASHBOARD, USER PROFILE, FORESTIERS, SERIE, DISTRIBUTION, LES FACTEURS DE RISQUES, ALERTE, and LISTE METEO ANNUELLE. The main area is divided into three sections. The top-left section, titled 'alerte', contains a map with a green polygon. The top-right section is a 'Données météo' form with fields for 'Année' and 'Mois' (dropdowns), and input fields for 'Température journalière (°C)', 'Humidité journalière (%)', 'Vitesse du vent annuelle (km/h)', 'Direction du vent annuelle', 'Précipitation annuelle (mm)', 'Humidité annuelle (%)', 'Température annuelle (°C)', 'Température minimale annuelle (°C)', and 'Température maximale annuelle (°C)'. Below these is a 'Type de prédiction' section with radio buttons for 'Classification' and 'Régression'. At the bottom of the form are 'Enregistrer' and 'Annuler' buttons. The bottom-right section shows a map with a green polygon and a tooltip displaying coordinates: 'X = 4089609.53', 'Y = 5124.86368', and a 'Prédire' button. A 'Log out' link is in the top right corner.

FIGURE 44 : Formulaire de prédiction météo et affichage du résultat sur la carte

3.9.0.5 Popups interactives et légendes

Lorsqu'un utilisateur clique sur un incendie ou un point de la carte, une popup s'affiche avec :

- Les coordonnées du point ;
- La valeur lue dans la couche raster active :
 - Soit la classe de risque prédite ;
 - Soit la surface brûlée estimée (en hectares).

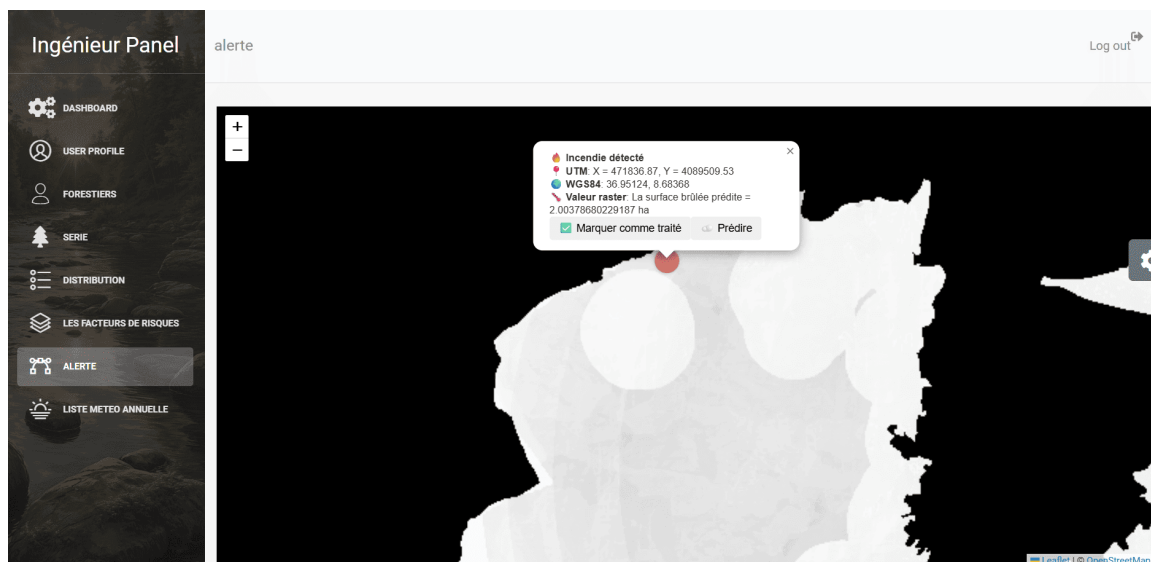


FIGURE 45 : Popup d'incendie avec valeur prédite et légende associée

Conclusion

Ce chapitre a présenté l'approche de modélisation appliquée à l'évaluation des risques d'incendie, depuis la construction des jeux de données jusqu'à l'intégration cartographique des résultats. L'interaction entre données spatiales, intelligence artificielle et visualisation permet de produire un outil de décision pertinent, à la fois prévisionnel (risques) et quantitatif (impacts).

Les résultats obtenus constituent une première base opérationnelle pour l'aide à la gestion territoriale, et peuvent être enrichis par l'intégration de données climatiques futures ou d'observations en temps réel.

DISCUSSION

Plan

1	Justification du modèle en cascade	25
2	Diagrammes de modélisation	25
3	Développement backend (Laravel)	36
4	Développement frontend (React & TerriaJS)	39
5	Extraction des valeurs raster pour la data science	41
6	Modélisation Data Science	45
7	Résultats de la modélisation	47
8	Intégration des résultats dans le web mapping	52
9	Visualisation des interfaces utilisateur	54

Introduction

Ce chapitre est dédié à l'analyse critique des résultats obtenus lors de la mise en œuvre du système d'évaluation des risques d'incendies forestiers. Nous commencerons par interpréter les performances des modèles prédictifs et les fonctionnalités développées, avant de discuter les limitations rencontrées au cours du projet. Enfin, nous proposerons des pistes d'amélioration pour optimiser les futures versions de la solution.

4.1 Analyse des résultats

Les résultats obtenus à l'issue du développement du système confirment la faisabilité d'une intégration cohérente entre des technologies web modernes (*React*, *Laravel*) et des modèles d'intelligence artificielle exposés via une API *Flask*. L'application finale permet :

- La saisie en temps réel des données météorologiques depuis une interface conviviale.
- La prédiction dynamique du risque d'incendie (**classification**) ou de la surface potentiellement brûlée (**régression**), en fonction des conditions saisies.
- L'affichage automatique de couches raster issues d'une base *PostgreSQL/PostGIS*, filtrées selon les résultats du clustering.
- La lecture de la valeur prédite directement sur la carte au clic, avec affichage dans une popup claire et accessible.

Le système s'est montré **réactif**, **modulaire**, et **extensible**. L'utilisation de l'API Flask pour déléguer les calculs de prédiction a permis d'isoler les traitements de *Data Science* sans alourdir le backend Laravel, tout en maintenant un temps de réponse satisfaisant (<1 seconde dans la majorité des tests).

La visualisation cartographique a également été optimisée grâce à *Leaflet*, facilitant la superposition des couches raster, routes, douars, et vigies, tout en garantissant une bonne lisibilité des données.

Enfin, la séparation logique entre les rôles (*forestier*, *ingénieur*) a permis de proposer une interface personnalisée à chaque utilisateur, répondant aux objectifs initiaux.

4.2 Limites de l'approche

Bien que l'application ait montré des résultats prometteurs, plusieurs limites ont été identifiées au cours du développement :

- **Données météo manuelles** : La saisie manuelle peut introduire des erreurs ou des imprécisions, impactant la fiabilité des prédictions.

- **Modèle statique** : Le modèle de clustering ne s'adapte pas aux nouvelles données. Toute mise à jour nécessite un réentraînement manuel.
- **Scalabilité limitée** : L'affichage de nombreuses couches raster ou l'accès simultané par plusieurs utilisateurs peut ralentir le système.
- **Exécution de scripts Python** : L'appel de scripts depuis Laravel présente des risques de sécurité (injection, surcharge) si mal encadré.
- **Dépendance aux données locales** : Les couches raster sont liées à des fichiers internes. Un découplage avec un serveur WMS améliorerait la flexibilité.

4.3 Pistes d'amélioration

Pour renforcer la fiabilité et la portée du système, plusieurs améliorations sont envisageables :

- **Automatiser la collecte météo** : Intégrer des services météo en ligne permettrait de fiabiliser les données et d'éviter la saisie manuelle.
- **Modèle adaptatif** : Développer un modèle capable d'être mis à jour automatiquement avec de nouvelles données améliorerait sa précision à long terme.
- **Serveur cartographique (WMS/WMTS)** : Utiliser un serveur dédié pour gérer les couches raster faciliterait leur ajout et amélioration dynamique.
- **Meilleure sécurité des scripts** : Encadrer l'exécution Python avec des API sécurisées ou des microservices asynchrones limiterait les risques.
- **Extensibilité nationale** : Le système peut être adapté à l'échelle de toute la Tunisie, au-delà de la région de Kroumirie (Jendouba), en intégrant des données raster couvrant l'ensemble du territoire forestier.

Conclusion

Cette discussion a permis de prendre du recul sur les résultats obtenus, en identifiant les forces mais aussi les faiblesses du système mis en place. Elle a mis en lumière les aspects à renforcer dans le futur pour faire de cette solution un outil encore plus performant et robuste dans le cadre de la prévention des incendies de forêts.

Conclusion générale

Ce projet de fin d'études, réalisé au sein de l'entreprise **MEDIANET**, a porté sur la conception et le développement d'une **application web intelligente dédiée à l'évaluation des risques d'incendies forestiers**, avec une application concrète dans la région de la Kroumirie, zone particulièrement vulnérable aux feux de forêt. Dans un contexte de changement climatique et de pression croissante sur les écosystèmes naturels, cette initiative s'inscrit dans une dynamique de préservation proactive du patrimoine forestier, en mobilisant les outils les plus avancés des technologies web, des systèmes d'information géographique (SIG) et de la data science spatiale.

La solution développée repose sur une architecture technologique robuste et cohérente, combinant **Laravel** pour la gestion du backend, **React** et **TerriaJS** pour la visualisation cartographique, **Python (Flask)** pour l'intégration des modèles prédictifs, et **PostgreSQL/PostGIS** pour la gestion des données spatiales. L'application embarque des algorithmes d'intelligence artificielle performants — tels que le *clustering K-Means*, le *Random Forest Classifier* et le *XGBoost Regressor* — permettant de modéliser finement les niveaux de risque et d'estimer les surfaces potentiellement impactées. Cette approche repose sur l'exploitation intelligente d'un large éventail de données climatiques, environnementales et topographiques, assurant une évaluation dynamique, fiable et adaptée aux réalités du terrain.

Les résultats obtenus démontrent la valeur ajoutée de l'outil : l'application permet aux gestionnaires forestiers d'accéder en temps réel à des cartes de risque actualisées selon les conditions météorologiques observées ou projetées, renforçant ainsi leur capacité d'anticipation, de prévention et de réaction face aux incendies. Par son interface ergonomique, pensée pour différents profils utilisateurs (visiteur, forestier, ingénieur, administrateur), la plateforme garantit une prise en main rapide et une intégration efficace dans les pratiques opérationnelles.

Au terme de ce projet, plusieurs perspectives d'évolution ont été identifiées afin d'assurer la pérennité et l'extension de la solution : automatisation de la collecte des données météorologiques via des API en temps réel, intégration de modèles d'apprentissage adaptatif pour améliorer la réactivité du système et mise en place de serveurs cartographiques spécialisés pour optimiser la gestion des couches raster.

En définitive, ce projet a abouti à la mise en place d'une solution technologique **fonctionnelle, innovante et durable**, illustrant la puissance de la convergence entre technologie et intelligence spatiale pour répondre aux enjeux environnementaux critiques. Il a également représenté une expérience

formatrice riche, nous dotant d'une **expertise solide en ingénierie logicielle, en géomatique et en gestion de projet**. L'application conçue constitue aujourd'hui une **base évolutive** et prometteuse pour le développement futur de systèmes intelligents de gestion des risques, au service de la protection des forêts tunisiennes.

backend=biber,style=plain,natbib=truebiblatex

Annexes

ANNEXE A : POSTGIS ET GESTION DES COUCHES SPATIALES

Plan

1	Analyse des résultats	61
2	Limites de l'approche	61
3	Pistes d'amélioration	62

Cette annexe présente l'utilisation de **PostGIS**, une extension spatiale de PostgreSQL, dans le cadre de la gestion, de l'analyse et de l'exploitation des données géospatiales. Elle détaille notamment l'importation de couches *raster* et *vecteur* dans la base de données, les outils utilisés, ainsi que les fonctions clés mobilisées au sein du projet.

A.1.1 Présentation de PostGIS

PostGIS enrichit PostgreSQL avec la prise en charge des objets géographiques. Il transforme une base relationnelle classique en un véritable entrepôt spatial, apte à gérer des entités cartographiques, les croiser, les interroger, et produire des analyses spatiales avancées.

Il permet de stocker, manipuler et interroger :

- Des données **vecteur** : points, lignes, polygones (villages, routes, unités forestières...),
- Des données **raster** : imageries ou couches continues (altitude, température, NDVI...),
- Des géométries 2D/3D, des projections diverses (SRID), des topologies complexes,
- Des **fonctions SQL spatiales** puissantes : `ST_Intersects`, `ST_Within`, `ST_Buffer`, `ST_Clip`, `ST_Value`, etc.

En intégrant PostGIS à l'architecture SIG du projet, nous avons pu uniformiser le traitement des données spatiales, garantir leur cohérence, et les rendre directement exploitables dans nos applications (cartes web, API REST, analyses Python).

A.1.2 Importation de rasters dans PostGIS

Les couches raster représentent des phénomènes continus (ex. : humidité, température, pente) sous forme matricielle. Leur intégration dans PostGIS permet d'interroger ces valeurs avec précision, de les croiser avec des objets vecteur (incendies, forêts), et de les exploiter dans des modèles prédictifs.

Nous utilisons l'utilitaire `raster2pgsql`, qui convertit les fichiers `.tif` en commandes SQL d'insertion, optimisées pour PostGIS.

```

saada@saafa MINGW64 ~
$ export PGBIN="/c/Program Files/PostgreSQL/17/bin"
export RASTER_DIR="C:\Users\saada\Downloads\rasters\rasters"
export PGPASSWORD="1234"

# Vérifiez d'abord que le chemin est correct
ls -la "$RASTER_DIR"

# Puis exécutez la boucle d'import
for f in "$RASTER_DIR"/*.tif; do
[ -f "$f" ] || continue
table_name=$(basename "$f" .tif | tr '[:upper:]' '[:lower:]' | tr -s ' ' '_' | tr -d '-')
echo "Importing $f as $table_name..."
"$PGBIN/raster2pgsql.exe" -s 32632 -I -C -Y -M "$f" public."${table_name}" | \
"$PGBIN/psql.exe" -h localhost -p 9191 -U postgres -d wildfires-map
done

```

FIGURE 46 : Script Bash automatisant l'importation des couches raster dans PostGIS

Explication des options utilisées :

- `-s 32632` : système de projection (ici WGS 84 / UTM zone 32N, utilisé pour la Tunisie).
- `-I` : création d'un index spatial sur le raster.
- `-C` : ajout automatique des contraintes de couverture.
- `-Y` : tuilage (division en blocs) pour optimiser les performances.
- `-M` : exclusion des tuiles entièrement nulles (gains d'espace).

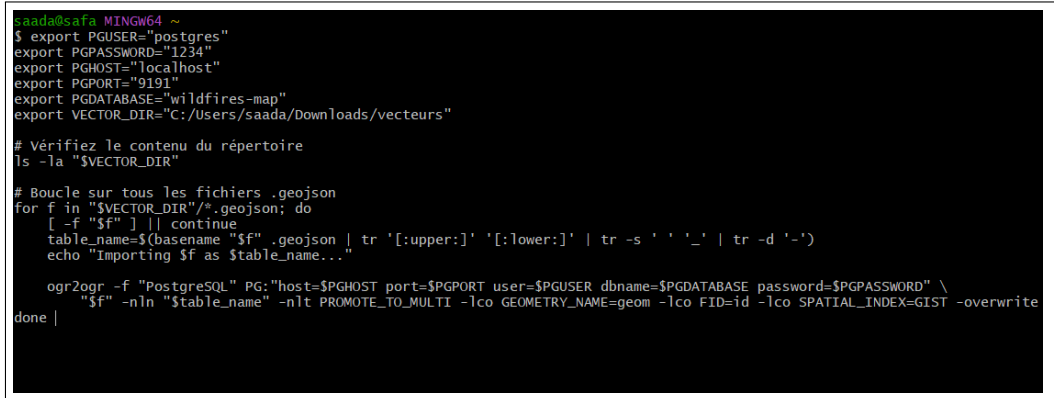
Une fois intégrés, les rasters deviennent interrogeables avec :

- `ST_Value(rast, geom)` : valeur du pixel sous une géométrie.
- `ST_SummaryStats(rast)` : statistiques globales (min, max, moyenne...).
- `ST_Clip(rast, geom)` : extraction d'une portion de raster selon une géométrie.
- `ST_Intersection` : croisement spatial raster/vecteur.

A.1.3 Importation de couches vecteur dans PostGIS

Les couches vecteur représentent des objets spatiaux discrets et structurés : limites administratives, séries forestières, zones à risque, infrastructures, etc. Elles sont essentielles dans l'analyse spatiale thématique, la cartographie interactive et la modélisation.

Nous utilisons l'outil `ogr2ogr` (inclus dans GDAL) pour importer efficacement des fichiers vecteur dans PostGIS. Ce processus est scripté pour automatiser l'import en masse de fichiers GeoJSON.



```
saada@saafa MINGW64 ~
$ export PGUSER="postgres"
export PGPASSWORD="1234"
export PGHOST="localhost"
export PGPORT="9191"
export PGDATABASE="wildfires-map"
export VECTOR_DIR="C:/Users/saada/Downloads/vecteurs"

# Vérifiez le contenu du répertoire
ls -la "$VECTOR_DIR"

# Boucle sur tous les fichiers .geojson
for f in "$VECTOR_DIR"/*.geojson; do
  [ -f "$f" ] || continue
  table_name=$(basename "$f" .geojson | tr '[:upper:]' '[:lower:]' | tr -s ' ' '_' | tr -d '-')
  echo "Importing $f as $table_name..."

  ogr2ogr -f "PostgreSQL" PG:"host=$PGHOST port=$PGPORT user=$PGUSER dbname=$PGDATABASE password=$PGPASSWORD" \
    "$f" -nln "$table_name" -nlt PROMOTE_TO_MULTI -lco GEOMETRY_NAME=geom -lco FID=id -lco SPATIAL_INDEX=GIST -overwrite
done |
```

FIGURE 47 : Script Bash pour l'importation automatique de couches vecteur (GeoJSON) dans PostGIS

Options clés de ogr2ogr :

- `-nln` : nom de la table cible dans la base.
- `-nlt PROMOTE_TO_MULTI` : uniformise les géométries (ex. : polygones simples → multipolygones).
- `-lco GEOMETRY_NAME=geom` : nom de la colonne géométrique (standardisé).
- `-lco SPATIAL_INDEX=GIST` : création automatique d'un index spatial.
- `-overwrite` : écrase la table si elle existe.

Avantages de PostGIS pour les vecteurs :

- Requêtes spatiales puissantes (croisements, sélections, agrégats...).
- Intégration homogène avec les rasters (via les fonctions spatiales).
- Interfaçage direct avec QGIS, Leaflet, GeoServer ou des API.
- Support de multiples formats : GeoJSON, Shapefile, GPKG, KML...

A.1.4 Exploitation des couches spatiales dans l'application

Une fois importées, les couches raster et vecteur sont :

- **Consultables via QGIS** pour la validation visuelle et l'analyse manuelle.
- **Servies via API REST (Laravel)** pour alimenter les cartes web et applications.
- **Requêtées dans des scripts Python** pour l'analyse prédictive (scikit-learn, geopandas...).
- **Visualisées dans Leaflet et TerriaJS** pour une cartographie interactive temps réel.
- **Exploitable dans les modèles** pour calculer les risques d'incendie en combinant des couches multiples.

PostGIS agit ainsi comme un **pivot central** entre les données brutes, les outils d'analyse, et les interfaces utilisateurs. Il garantit une gestion performante, sécurisée et évolutive des données spatiales dans toute l'architecture SIG du projet.

A.1.5 Exportation de données spatiales depuis PostGIS

PostGIS permet non seulement l'importation de données géospatiales, mais aussi leur export vers de nombreux formats standards : GeoJSON, Shapefile, KML, ou encore CSV (avec coordonnées).

Cette capacité est particulièrement utile pour :

- Générer des jeux de données à partager ou archiver,
- Réutiliser les résultats d'analyse dans des outils externes (QGIS, outils Python, etc.),
- Créer des exports dynamiques dans une API (GeoJSON pour Leaflet ou TerriaJS).

Exemple d'export vers GeoJSON avec ogr2ogr :

Listing A.1 – Export d'une couche vecteur PostGIS en GeoJSON

```
ogr2ogr -f "GeoJSON" output/villages_export.geojson \  
PG:"host=localhost dbname=wildfires-map user=postgres password=1234 port=9191" \  
-sql "SELECT * FROM villages WHERE zone_risque = 'leve'"
```

Autres formats pris en charge :

- Shapefile (.shp) : pour les échanges avec des institutions SIG traditionnelles.
- CSV : pour les exports simples ou statistiques.
- KML : pour visualiser les données dans Google Earth.

Bonnes pratiques lors de l'export :

- Toujours spécifier une projection de sortie (`-t_srs EPSG : 4326 pour les applications web`), Utiliser des requêtes SQL.
- Vérifier la cohérence des géométries et des attributs exportés.

Ainsi, l'export des données PostGIS constitue une étape essentielle pour la diffusion, la réutilisation et la valorisation des résultats de l'analyse spatiale.

ANNEXE B :REQUÊTES SPATIALES SQL

UTILISÉES DANS LE PROJET

Cette annexe présente une sélection représentative de requêtes SQL spatiales utilisées dans l'implémentation du système backend (Laravel + PostGIS). Ces requêtes ont été mobilisées dans les modules d'insertion, de visualisation, d'analyse, et d'enrichissement des données spatiales.

B.1.1 Insertion d'une entité avec géométrie (ST_SetSRID)

Chaque incendie ajouté est géolocalisé via un point géométrique (x/y) projeté en EPSG :32632 :

Listing B.1 – Insertion d'un point avec géométrie

```
INSERT INTO dgf (x, y, temperature_journaliere, humidite_journaliere, Date_, geom)
VALUES (9.2531, 36.8341, 33.2, 14.0, '2025-06-01',
ST_SetSRID(ST_MakePoint(9.2531, 36.8341), 32632));
```

B.1.2 Export GeoJSON depuis PostGIS (ST_AsGeoJSON)

Les entités vecteur (ex. : séries forestières) sont converties en GeoJSON pour la visualisation web :

Listing B.2 – Export GeoJSON depuis PostGIS

```
SELECT
    id,
    "NOM_SERIE",
    "SUBD",
    "NOM_TRIAGE",
    "SURFACE",
    "PERIMETRE",
    ST_AsGeoJSON(ST_Transform(geom, 4326))::json AS geometry
FROM
    series_a_accorde;
```

B.1.3 Fusion multi-sources d'incendies (UNION ALL)

Vue unifiée pour lister les incendies récents (DGF, NASA, EFFIS) :

Listing B.3 – Fusion de sources dans une vue unifiée

```
SELECT id, 'dgf' AS source, (x::text || ', ' || y::text) AS location, Date_ AS date, SUP AS value
FROM dgf
```

```
UNION ALL
SELECT id, 'effis', (X::text || ', ' || Y::text), initialdat, area_ha
FROM effis_final
UNION ALL
SELECT id, 'nasa', (x::text || ', ' || y::text), ACQ_DATE, BRIGHTNESS
FROM nasa_final
ORDER BY date DESC
LIMIT 10;
```

B.1.4 Encodage raster en GeoTIFF (ST_AsTIFF)

Pour générer une image GeoTIFF à la volée depuis une couche raster PostGIS :

Listing B.4 – Encodage TIFF pour réponse HTTP

```
SELECT encode(ST_AsTIFF(rast), 'base64') AS img
FROM prediction_cluster_0_biclass
LIMIT 1;
```

Cette image est ensuite renvoyée via une API Laravel avec les bons en-têtes MIME.

B.1.5 Traitement externe Python (enrichissement spatial)

Après insertion, un script Python est déclenché pour enrichir l'incendie avec des valeurs raster proches :

Listing B.5 – Appel d'un script Python d'enrichissement

```
python scripts/enrich_dgf_one.py 42
```

Ce script effectue des requêtes spatiales type `'ST_Value'`, `'ST_SummaryStats'`, `'ST_Clip'` sur la base PostGIS.

Conclusion

Les requêtes SQL spatiales constituent un pilier essentiel dans l'automatisation et l'intelligence de la plateforme. Elles permettent de :

- Convertir des données brutes en GeoJSON lisibles,
- Réaliser des jointures ou agrégations spatiales complexes,

- Gérer dynamiquement les couches raster,
- Alimenter les processus d’analyse et de prédiction.

Grâce à PostGIS, la base de données devient un moteur spatial puissant, central dans toute l’architecture du système.

ANNEXE C :INTÉGRATION DU MODÈLE DE PRÉDICTION VIA API FLASK

Cette annexe décrit l'intégration technique entre le backend Laravel du projet et une API Flask Python dédiée à la prédiction de clusters climatiques à l'aide d'un modèle de machine learning préalablement entraîné et sauvegardé.

C.1.1 Objectif de l'intégration

L'objectif est de fournir un service permettant, à partir d'un ensemble de variables météorologiques (température, humidité, vent, précipitations...), de déterminer dynamiquement le **cluster climatique** auquel appartient une observation donnée. Ce cluster est ensuite utilisé pour filtrer des couches raster correspondantes.

C.1.2 Architecture du système

- Le modèle d'apprentissage automatique est encapsulé dans une API Flask, exposant une route POST `/predict`.
- L'application Laravel envoie les données météorologiques via une requête HTTP POST à cette API.
- L'API retourne le numéro de cluster, qui est utilisé par Laravel pour charger les couches raster correspondantes.

C.1.3 Code côté Flask (Python)

Le serveur Flask charge un modèle `'scikit-learn'` (fichier `'.pkl'`) et expose une route qui reçoit un dictionnaire JSON avec les features. Ces valeurs sont injectées dans le modèle pour retourner le cluster.


```

4
5 app = Flask(__name__)
6
7 # Charger le modèle depuis le fichier .pkl
8 model = joblib.load("meteo_cluster_model.pkl")
9
10 @app.route('/predict', methods=['POST'])
11 def predict():
12     data = request.get_json()
13     # Extraire les features dans le bon ordre attendu par le modèle
14     features = [
15         data['temperature_journaliere'],
16         data['humidite_journaliere'],
17         data['vitesse_vent_annuelle'],
18         data['direction_vent_annuelle'],
19         data['precipitation_annuelle'],
20         data['humidite_annuelle'],
21         data['temperature_annuelle'],
22         data['temperature_minimale_annuelle'],
23         data['temperature_maximale_annuelle'],
24         data['annee'],
25         data['mois']
26     ] # Prédire le cluster
27     prediction = model.predict([features])
28
29     return jsonify({'cluster': int(prediction[0])})
30
31 if __name__ == '__main__':
32     app.run(host='127.0.0.1', port=5000)
33

```

FIGURE 48 : Serveur Flask – prédiction de cluster

C.1.4 Code côté Laravel (PHP)

Depuis Laravel, une requête est envoyée vers le serveur Flask avec les données météorologiques issues d'un formulaire ou d'une base de données. En cas de succès, le cluster est extrait de la réponse et utilisé pour filtrer les couches raster.

```

public function getClusterFromFlask(Request $request)
{
    $categorie = $request->categorie;
    $response = Http::post('http://127.0.0.1:5000/predict', [
        'temperature_journaliere' => $request->temperature_journaliere,
        'humidite_journaliere' => $request->humidite_journaliere,
        'vitesse_vent_annuelle' => $request->vitesse_vent_annuelle,
        'direction_vent_annuelle' => $request->direction_vent_annuelle,
        'precipitation_annuelle' => $request->precipitation_annuelle,
        'humidite_annuelle' => $request->humidite_annuelle,
        'temperature_annuelle' => $request->temperature_annuelle,
        'temperature_minimale_annuelle' => $request->temperature_minimale_annuelle,
        'temperature_maximale_annuelle' => $request->temperature_maximale_annuelle,
        'annee' => $request->annee,
        'mois' => $request->mois
    ]);

    if ($response->successful()) {
        $cluster = $response->json()['cluster'];
        $rasterLayers = $this->getRastersForCluster($cluster, $categorie);
        return response()->json([
            'success' => true,
            'cluster' => $cluster,
            'rasters' => $rasterLayers,
        ]);
    }

    return response()->json(['success' => false, 'message' => 'Erreur API Flask'], 500);
}

```

FIGURE 49 : Requête vers Flask et récupération du cluster

C.1.5 Résultat et intégration finale

Grâce à ce système :

- Le calcul de prédiction est délégué à une API spécialisée, indépendante du cœur Laravel.
- La modularité permet de réentraîner et réinjecter un nouveau modèle facilement.
- L'interface utilisateur peut déclencher en temps réel la classification climatique d'un jeu de données.

Cette approche garantit une séparation claire des responsabilités : traitement scientifique côté Python, logique métier et présentation côté Laravel.

—

Remarque : Cette API pourrait être étendue pour retourner également des scores de probabilité, ou pour recevoir des jeux de données complets (batch), si nécessaire.

يهدف هذا المشروع، المنجز في شركة MEDIANET، إلى تطوير تطبيق ويب ذكي لتقييم المخاطر المرتبطة بالحرائق الغابية في منطقة خميرية شمال تونس، اعتماداً على بيانات جغرافية مكانية متعددة المصادر. يعتمد النظام على بنية تقنية متكاملة تشمل Laravel للواجهة الخلفية، و React و TerriaJS/Leaflet للعرض التفاعلي، و PostgreSQL/PostGIS لتخزين البيانات المكانية. تم دمج نماذج من علم البيانات مثل K-Means و Random Forest و XGBoost عبر API Flask، ما يتيح للمستخدمين، خصوصاً المهندسين، إدخال بيانات مناخية والحصول على توقعات ديناميكية لمستوى الخطر أو للمساحة المحتمل احتراقها، وذلك بشكل مرئي وتفاعلي على الخريطة. يساهم هذا التطبيق في دعم اتخاذ القرار وإدارة الحرائق بفعالية.

كلمات مفاتيح : حرائق الغابات، نظم المعلومات الجغرافية، تطبيق ذكي، التنبؤ بالمخاطر، خميرية

Résumé

Le présent projet, réalisé au sein de l'entreprise **MEDIANET**, consiste en la conception d'une application web intelligente pour l'évaluation et la visualisation des risques d'incendies forestiers dans la région de la Kroumirie (Tunisie), à partir de données géospatiales hétérogènes. L'architecture technique repose sur **Laravel** (backend), **React/TerriaJS/Leaflet** (frontend), et **PostgreSQL/PostGIS** pour la gestion spatiale. Des modèles de data science (*K-Means*, *Random Forest*, *XGBoost*) sont intégrés via une API **Flask**. L'application permet aux utilisateurs, notamment les ingénieurs, de saisir des données météorologiques pour obtenir des prédictions dynamiques de la classe de risque ou de la surface brûlée, visualisées sur une carte interactive. Cette solution constitue un véritable outil d'aide à la décision en matière de gestion des incendies.

Mots clés : incendies, SIG, application web, prédiction, Kroumirie

Abstract

This project, carried out at **MEDIANET**, aimed to develop an intelligent web application for assessing and visualizing forest fire risks in the Kroumirie region (Tunisia) using heterogeneous geospatial data. The system relies on a robust architecture combining **Laravel** (backend), **React/TerriaJS/Leaflet** (frontend), and **PostgreSQL/PostGIS** for spatial data management. Data science models such as *K-Means*, *Random Forest*, and *XGBoost* are exposed through a **Flask API**. The application enables users, especially engineers, to input meteorological data and obtain dynamic predictions of fire risk classes or potentially burned areas, displayed interactively on the map. It serves as a valuable decision-support tool for forest fire management.

Keywords : forest fires, GIS, smart application, prediction, Kroumirie